

# 成本不對稱下以品質為約束的 可恢復階層式 KV State 管理

Cost-Asymmetric Hierarchical KV State Management  
for Long-Context LLM Inference

(作者資訊於投稿時填入)

## Abstract

長上下文 (long-context) LLM 推論的核心系統瓶頸之一，是 KV cache 隨上下文長度線性成長並耗盡 GPU 高頻寬記憶體 (HBM)。既有工作已分別處理了這個問題的各個面向：可恢復的驅逐、學習式的未來效用預測、重算與傳輸的取舍，以及以品質為約束的壓縮。然而，據我們所知，尚無工作將這些決策整合到單一的線上決策之中。

本文的出發點是一個橫跨硬體的量化事實：重算與傳輸的成本比  $\kappa$  隨平台變動達 32 倍——於 AMD Instinct MI300X (192 GB HBM3) 為 6–21 倍，於 NVIDIA RTX 3090 (24 GB GDDR6X) 則為 54–190 倍。此差異使「單一套 KV 放置策略適用於所有硬體」的隱含前提不成立。

問題的性質同樣隨平台改變。在 MI300X 上，8B 至 14B 級模型於 128K 上下文的單請求 KV footprint 僅佔 HBM 的 8–10%，Llama-3.1-8B 甚至可在單卡容納 1M token 的上下文 (需 YaRN 外推)；此處的壓力來自多租戶下的機會成本，而非容量本身。在 24GB 級的加速器上，單請求上下文本身即構成容量壓力，既有文獻的經典設定依然成立。兩者皆為真實的部署情境，而最佳策略在二者上並不相同。

基於此，我們將 KV 放置形式化為一個受品質約束的序列決策問題，其動作空間包含 GPU 全精度、GPU FP8、GPU INT4、CPU、SSD 與「丟棄後重算」六種狀態，並在「品質退化不超過  $\epsilon$ 」的硬性約束下，聯合最佳化未來效用、傳輸成本、重算成本與記憶體壓力。我們特別論證：重算的價值不在於單次事件的成本 (其代價比傳輸高 6–21 倍於 MI300X，54–190 倍於 RTX 3090)，而在於它在被需要之前不佔用任何資源；其正確性因而完全取決於未來效用預測的品質。

我們設計了雙平台實驗 (RTX 3090 的 30:1 與 MI300X 的 84:1，相距 2.8 倍)，使硬體條件化的主張成為可證偽的。二者並非此光譜的兩端——GH200 的 9:1 更低，MI355X 的 127:1 更高——但構成我們實際可取得的最大跨度。**本文為初稿**。第 6 節給出完整的實驗計畫與消融設計，實驗結果尚未取得。

**Keywords:** LLM inference; KV cache; memory hierarchy; long context; quality-constrained optimization; cost-sensitive learning; AMD MI300X; NVIDIA RTX 3090; vLLM

## 1 Introduction

Transformer 模型在自回歸生成時，會將每個已處理 token 的 key 與 value 向量保存下來，避免在生成每個新 token 時重算整段前綴。這組中間狀態即 KV cache，是以記憶體

換取計算時間的經典設計。其代價是記憶體佔用隨上下文長度線性成長：對 Llama-3.1-8B (32 層、8 個 KV head、head dimension 128、BF16)，每個 token 需要 128 KiB，1M token 的上下文即需要 122 GB。在 80GB 級的加速器上，這使得系統的限制因素從計算能力轉移到記憶體容量。

圍繞此問題的研究已密集展開。既有工作大致沿四條軸線展開：(i) **分層儲存**，將 KV 卸載到 CPU DRAM 或 SSD [17, 25, 28, 37, 39]；(ii) **可恢復驅逐**，使被降級的 KV 在重新變得重要時能夠召回 [10, 19]；(iii) **學習式未來效用預測**，以訓練得到的策略取代手寫啟發式規則 [4, 11, 34]；(iv) **重算與傳輸的取舍** [18, 21, 23]。此外，KVserve [30] 首次將模型品質寫為壓縮決策的硬性約束，而 Fang 等人 [14] 首次將分層 KV 放置形式化並推導理論上界。

**本文的出發點：一個不再成立的前提。** 上述工作幾乎全部在 NVIDIA A100/H100 級硬體 (40–80 GB HBM) 上評估。我們的量化分析 (第 2 節) 顯示，在 AMD Instinct MI300X (192 GB HBM3) 上，8B–14B 級模型於 128K 上下文下的單請求 KV footprint 僅 15.6–19.5 GB，佔 HBM 不到 10%。在此類硬體上，既有文獻用以立論的「單請求容量壓力」場景幾乎不存在。

這並不代表問題消失，而是問題的性質改變了。在 192 GB 的 HBM 上，一個 session 的冷 KV 所佔據的每 GB，都是其他請求無法用來擴大批次的容量。問題因此從「放不下，該驅逐什麼」轉變為「放得下，但該保留嗎」，成為一個機會成本問題，而機會成本只在多租戶場景中存在。

**第二個觀察：卸載在大容量 HBM 硬體上更昂貴。** MI300X 的 HBM 頻寬 (5.3 TB/s 峰值) 與主機鏈路頻寬 (PCIe Gen5  $\times 16$ ，單向理論 63.0 GB/s) 之比為 84:1，而 H100 為 52:1，MI355X 更達 127:1。此比值持續惡化，因為 HBM 容量與頻寬逐代成長，而主機鏈路自 MI300X 至 MI355X 完全未變。灌滿或抽乾 MI300X 上 Llama-3.1-8B 的 157.8 GiB KV 預算，經 PCIe 需 2.69 秒，而在 HBM 內僅需 32 毫秒。此比值必然等於頻寬比 84 倍。

貢獻。

1. 我們量化了重算對傳輸的成本比  $\kappa$  隨平台變動達 32 倍 (MI300X 上 6–21 倍、RTX 3090 上 54–190 倍)，並指出正確的判準是 FLOPS 對主機鏈路之比而非 HBM 頻寬比。同一組量化分析亦顯示，既有 KV 卸載文獻的容量前提在大容量 HBM 加速器上不成立，

問題性質由容量轉為機會成本，而在 24 GB 級加速器上則仍然成立（第 2 節）。

2. 我們系統性地比對了 15 篇近期工作，指出四項成本（未來效用、傳輸成本、重算成本、記憶體壓力）從未在單一線上決策中同時出現；現有最高覆蓋為 Marconi [35] 的三項，其因僅在 GPU 上運作而結構性地缺少傳輸成本（第 3 節）。
3. 我們將 KV 放置形式化為受品質約束的序列決策問題，動作空間包含六種狀態且明確納入「丟棄後重算」；並論證重算的價值來自其資源佔用特性而非單次成本，因而其正確性取決於未來效用預測（第 4 節）。
4. 我們提出 Tiara 的設計，透過 vLLM 的 CachePolicy 介面實作，可在 CUDA 與 ROCm 上零程式碼修改部署（ROCm 上的執行期限限制見第 7 節）。
5. 我們指出既有學習式 KV 管理方法所用的對稱損失函數，在動作成本相差 6–190 倍（隨平台而異）的設定下失準（10 倍不對稱下，Bayes 最佳門檻自  $p = 0.5$  移至  $p \approx 0.09$ ），並提出將動作成本納入損失的兩種形式。經比對 8 個既有學習式系統，尚無工作將異質動作成本編碼進 KV 管理的損失函數（第 5.3 節）。
6. 我們給出完整的實驗計畫，包含以成本項為單位（而非以模組為單位）的消融設計，以及一個離線最優 oracle 作為 go/no-go 判準（第 6 節）。

**本稿狀態。** 本文為初稿。第 2 節的量化分析與第 3 節的文獻比對均已完成並經查證；第 6 節為實驗計畫，尚未執行。我們在第 7 節誠實說明此一狀態的意涵。

## 2 Background and Motivation

### 2.1 KV cache 的記憶體佔用

對採用 grouped-query attention (GQA) 的模型，每 token 的 KV 佔用為

$$b_{\text{tok}} = 2 \times L \times H_{kv} \times d_h \times s \quad (1)$$

其中  $L$  為層數， $H_{kv}$  為 KV head 數， $d_h$  為 head dimension， $s$  為每元素位元組數，前導的 2 對應 K 與 V。

表 1 給出代表模型的實際數值。所有組態均直接取自 HuggingFace config.json。

**權重精度是正交的部署參數。** 本文將 LLM 權重視為 frozen，且不修改其精度。式 (4) 的動作空間描述的是 **KV cache 的儲存精度**，與模型權重的精度是兩個獨立的維度：

| 維度           | 取值                           |
|--------------|------------------------------|
| 權重精度（非本文變數）  | BF16 / FP8 / INT8 / AWQ-INT4 |
| KV 狀態（本文的決策） | 式 (4) 的六個動作                  |

我們因此不假設任何特定權重精度為「主流」。實驗在代表性的部署配置下進行——平台 A 採 AWQ-INT4（記憶

**Table 1:** KV footprint 與原生上下文長度。所有組態均經 HuggingFace config.json 直接驗證（2026-08-29）。原生 ctx 欄是選模的硬條件：低於評測長度者必須施加 RoPE 外推，而外推本身會降低品質，混淆  $\epsilon$  的量測。 $M$  為 MoE。

| 模型                         | $L$ | $H_{kv}$ | $d_h$ | KV/tok       | 原生 ctx            |
|----------------------------|-----|----------|-------|--------------|-------------------|
| Qwen2.5-7B-Inst-1M         | 28  | 4        | 128   | <b>56 Ki</b> | <b>1,010,000</b>  |
| Llama-3.1-8B-Inst          | 32  | 8        | 128   | 128 Ki       | 131,072           |
| Qwen3-8B                   | 36  | 8        | 128   | 144 Ki       | 40,960            |
| Qwen3-14B                  | 40  | 8        | 128   | 160 Ki       | 40,960            |
| Qwen3-30B-A3B <sup>M</sup> | 48  | 4        | 128   | 96 Ki        | 40,960            |
| Llama-4-Scout <sup>M</sup> | 48  | 8        | 128   | 192 Ki       | <b>10,485,760</b> |

**Table 2:** 單請求 KV footprint 與容納情形 (BF16 權重 + BF16 KV)。✓ 表示權重加 KV 可置入。§ 超出原生 max\_position\_embeddings，需 YaRN 外推（見第 6 節）。

| 模型            | ctx                   | KV            | H100<br>80 GB | MI300X<br>192 GB |
|---------------|-----------------------|---------------|---------------|------------------|
| Llama-3.1-8B  | 128K                  | 15.6 GB       | ✓             | ✓                |
|               | 256K                  | 31.2 GB       | ✓             | ✓                |
|               | <b>1M<sup>§</sup></b> | <b>122 GB</b> | ×             | ✓                |
| Qwen2.5-7B-1M | 256K                  | 13.7 GB       | ✓             | ✓                |
|               | 1M                    | 53.4 GB       | ✓             | ✓                |
| Llama-3.1-70B | 128K                  | 39.1 GB       | ×             | ×                |

體受限的消費級 GPU 之常見配置)，平台 B 採 BF16 與 FP8——並將權重精度的影響列為敏感度分析（第 6 節），而非主要貢獻。將兩個維度混為一談會使本文同時變成權重量化、KV 卸載、驅逐與重算四個研究問題，模糊真正的貢獻。

**關於 BF16 與 FP16。** 本文所檢視的六個模型其 config.json 的 torch\_dtype 均為 bfloat16。二者皆為 2 位元組，故式 (1) 的算術不變，但本文一律使用 **BF16** 以與實際發布的權重一致。

### 2.2 觀察一：單請求容量壓力在 MI300X 上幾乎不存在（但在 24 GB 級加速器上成立）

表 2 比較同一組模型在 H100 (80 GB) 與 MI300X (192 GB) 上的容納能力。此處固定為 **BF16 權重** 以隔離變因（權重精度的影響見上文與敏感度分析），並保留 10% 記憶體作為 activation 與碎片化餘裕。平台 A 的實際實驗採 AWQ-INT4 權重，其容納能力見第 2.6 節。

三個結論值得強調：

(a) 在 MI300X 上，8B–14B 模型於 128K 上下文僅消耗 HBM 的 8–10%。既有文獻普遍採用的「單請求、上下文遞增」實驗設定，在此硬體上量測不到記憶體壓力。相對地，既有的卸載瓶頸量測 [33] 顯示在 80 GB 級硬體上 99% 的延遲花在傳輸，但該結論建立在容量壓力存在的前提下。

(b) Llama-3.1-8B 的 1M token 上下文 (122 GB KV 加 16 GB 權重) 可完整置入單張 MI300X，而 H100 無法。若啟用 MI300X 原生支援的 FP8，Llama-3.1-70B 的 256K 上

**Table 3:** 記憶體階層比值與計算-傳輸比。規格取自 [3,9]。主機鏈路為單向理論值；FLOP/byte 為 dense BF16 峰值除以單向鏈路頻寬，代表一位元組穿越鏈路的時間內可 retire 的算術運算量。

|                        | Mem BW    | Host link | Mem:link     | FLOP/byte     |
|------------------------|-----------|-----------|--------------|---------------|
| RTX 3090 <sup>†‡</sup> | 0.94 TB/s | 31.5 GB/s | <b>30:1</b>  | 2,254         |
| H100 SXM               | 3.35 TB/s | 64.0 GB/s | 52:1         | 15,459        |
| H200 SXM               | 4.8 TB/s  | 64.0 GB/s | 75:1         | 15,459        |
| MI300X                 | 5.3 TB/s  | 63.0 GB/s | <b>84:1</b>  | <b>20,752</b> |
| MI355X                 | 8.0 TB/s  | 63.0 GB/s | <b>127:1</b> | —             |
| GH200                  | ~4 TB/s   | 450 GB/s  | ~9:1         | —             |

<sup>†</sup> RTX 3090 使用 GDDR6X 而非 HBM，PCIe 4.0×16。<sup>‡</sup> RTX 3090 之值由 328 TC × 1,695 MHz × 128 FLOP/clock = 71.2 TFLOPS 除以 31.5 GB/s 得出 [1]。GA10x 的 tensor core 於 FP32 累加下為 128 FLOP/clock (FP16 累加為 256)，BF16 與 FP16-FP32 累加同速。此處易誤取 35.6 TFLOPS——該值對應 64 FLOP/clock，為 TF32 tensor 或非 tensor FP32 的峰值，與 BF16 GEMM 的資料路徑不同。

下文亦可置入。

(c) 在 MI300X 上，真實壓力來自三處：並行度 (Llama-3.1-8B 在 BF16 權重下約 10 個並行 128K 請求即耗盡 HBM，FP8 權重下約 21 個)、更大的模型、以及跨 session 的 KV 保留 (唯一無上界的需求來源)。前二者是靜態的容量問題，第三者才是真正的策略問題。在記憶體較小的加速器上 (如本文的平台 A，24 GB) 則存在第四個來源：單請求上下文本身 (見第 2.6 節)。

## 2.3 觀察二：卸載的相對代價隨硬體世代惡化

表 3 給出關鍵比值。

比值持續惡化的原因是結構性的：HBM 容量與頻寬逐代提升，而 PCIe Gen5 ×16 自 MI300X 至 MI355X 完全未變。其後果是，任何卸載策略都必須掩蓋一個比 HBM 存取高近兩個數量級的延遲。

GH200 的 9:1 是例外：藉由 NVLink-C2C 的 450 GB/s，其卸載經濟性優於任何 x86 主機體的加速器約 7–14 倍。同一套 KV 放置策略不可能同時在 9:1 與 127:1 的平台上最佳，這是硬體條件化策略的直接論據。

## 2.4 觀察三：重算的價值不在單次成本

決定「重算或傳輸」的正確指標並非 HBM 頻寬比，而是 FLOPS 對主機鏈路之比，即一個位元組穿越 PCIe 的時間內，加速器能 retire 多少算術運算。MI300X 的 dense BF16 峰值為 1307.4 TFLOPS，故每傳輸一位元組可換得約 20,750 FLOP；H100 為 989.4 TFLOPS，對應 15,460 FLOP。MI300X 在此指標上優於 H100 約 1.34 倍。

然而，在絕對值上傳輸仍然較便宜，且便宜的程度隨平台劇烈變動。以簡化的 prefill 模型 (線性層約  $2N_{\text{params}}$  FLOP/token，100% MFU) 估算，成本比  $\kappa = t_{\text{recompute}}/t_{\text{transfer}}$  如表 4。

且重算成本隨 block 的絕對位置增長，而傳輸成本則保持恆定。表 4 的數值應視為  $\kappa$  的下界。

此處須區分兩個容易混淆的量。整段 prefill 的總成本確為序列長度的二次式 (attention 為  $O(L^2)$ )；但本文的動作空間作用於固定大小的 block，而在位置  $P$  重算一個大小為  $C$  的 block，其 attention 需讀取前序的  $P$  個 token，成本為  $O(C \cdot P)$ ——對  $P$  為線性。我們在平台 A 上以

**Table 4:** 重算對傳輸的成本比  $\kappa$ 。 $\kappa$  跨平台變動達 32 倍，這是本文「硬體條件化策略」主張最直接的量化依據。

| 平台       | 模型            | 重算            | 傳輸           | $\kappa$    |
|----------|---------------|---------------|--------------|-------------|
| MI300X   | Llama-3.1-8B  | 12.2 $\mu$ s  | 2.08 $\mu$ s | <b>6×</b>   |
|          | Llama-3.1-70B | 107 $\mu$ s   | 5.20 $\mu$ s | <b>21×</b>  |
| RTX 3090 | Llama-3.1-8B  | 225 $\mu$ s   | 4.16 $\mu$ s | <b>54×</b>  |
|          | Llama-3.1-70B | 1,972 $\mu$ s | 10.4 $\mu$ s | <b>190×</b> |

$C = 2048$ 、 $P \in \{0, 4K, 8K, 16K, 24K\}$  實測 Llama-3.1-8B (每點三次取中位數)，得

$$C_{\text{recompute}}(P) = 513.0 \text{ ms} + 26.9 \text{ ms} \times \frac{P}{1000}, \quad (2)$$

線性擬合的最大偏差為 1.6%，五個量測點皆落於直線上，未觀察到二次成分。自  $P = 0$  至  $P = 24,576$  成本成長 2.29 倍。此係數依平台與模型而定，但線性這個形式來自  $O(C \cdot P)$ ，與平台無關。將重算視為單一常數會在長上下文下系統性低估其成本。

$\kappa$  亦隨模型架構變動，且方向可事先計算。 $\kappa$  的分子由啟用參數量決定，分母由 KV 大小決定，而 MoE 使兩者解耦：其 KV 由 attention 層決定 (與同規模 dense 模型相同)，但重算僅需啟用的專家。以 Qwen3-30B-A3B 為例 (48 層、4 個 KV head、128 個專家中啟用 8 個，啟用參數約 3B)，其  $\kappa$  在 RTX 3090 上為 27，僅為同平台 dense 模型 (48–108) 的約一半。

這產生一個可證偽的預測：在固定平台上，MoE 模型的最佳 Drop 觸發門檻應高於 dense 模型。第 6 節將檢驗之。此軸與硬體軸正交，兩者共同構成本文「成本結構決定策略」主張的雙重檢驗。

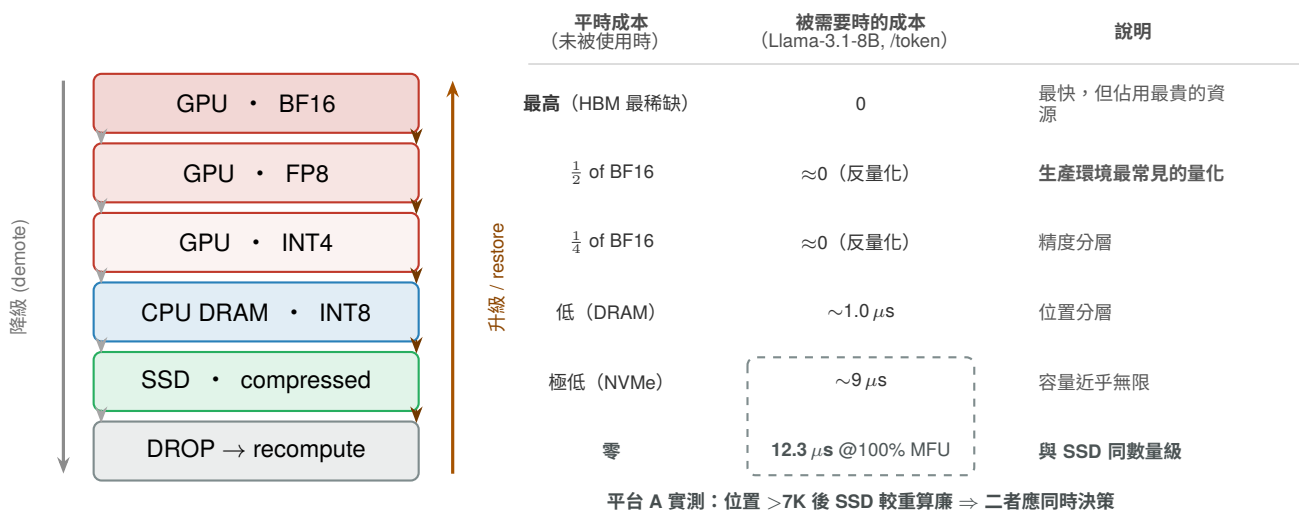
圖 1 揭示了一個關鍵的非對稱性：重算的價值不在於其單次事件成本，而在於它在被需要之前不佔用任何資源。因此重算的正確條件是「該 KV block 未來被存取的機率足夠低」——低到節省的容量與頻寬能夠抵償萬一被需要時 6–190 倍的代價。

這個推論的重要後果是：未來效用預測不是系統的附加元件，而是決定重算動作正確與否的唯一依據。這與第 3 節的文獻分析指向同一結論。

## 2.5 觀察四：prefill 與 decode 的卸載經濟性相差五個數量級

同一筆搬移在 prefill 攤提於整段前綴，在 decode 則每產生一個 token 就重付一次。這個不對稱決定了動作空間的哪些層在何種工作負載下可用，而既有工作多半只在其側成立。

兩個階段的瓶頸不同。prefill 一次處理  $N$  個 token，算術強度高，受限於算力；decode 一次處理一個 token，但每一步都須讀取整個 KV cache，算術強度趨近 1，受限於記憶體頻寬。我們在平台 A 上實測此差距 (Llama-3.1-8B 與 Qwen2.5-7B-1M，AWQ 權重，四個上下文長度)：



**Figure 1:** Tiara 的動作空間與其成本結構。與既有工作的差異有二：(i) 位置分層 (GPU/CPU/SSD) 與精度分層 (BF16/FP8/INT4/INT8) 被統一為單一階梯，而既有工作只做其中之一；(ii) 「丟棄後重算」是階梯上的一層，而非檢索失敗後的補救。此六階為典型配置之示意，量化器可替換（見第 4 節）。關鍵的非對稱性在「平時成本」欄：**重算在被需要之前不佔用任何資源**，這是它唯一的優勢，也是它唯一的正當理由。右下虛線框標示一個常被忽略的事實：自 SSD 載入與重算處於同一數量級，兩者本就應在同一個決策中被比較。此點已由平台 A 的端到端量測證實，但量級與歸因均與純頻寬推算不同——以 Llama-3.1-8B 於 16K 上下文計，自磁碟階取回為  $346 \mu s / token$ 、重算為  $326 \mu s / token$ （皆已扣除 GPU 命中的基準），確為同一數量級，然其絕對值較「 $\sim 9 \mu s / token$  @ 7 GB/s」的理想頻寬推算高出約 30 倍。分解後可見磁碟 I/O 僅佔搬運成本的約 35%，其餘為分層機制的查表、cascade 排程與 promotion 開銷；將裝置由 SATA SSD（實測 0.49 GB/s）換為 NVMe（1.65 GB/s，頻寬 3.4 倍）後端到端成本未降反升。詳見第 6 節。

| 上下文     | prefill (ms/tok) | decode (ms/tok) | 倍數  |
|---------|------------------|-----------------|-----|
| 16,384  | 0.316            | 17.0            | 54× |
| 65,536  | 0.591            | 23.5            | 40× |
| 126,976 | 0.903            | 29.7            | 33× |
| 258,048 | 1.264            | 29.4            | 23× |

此差距使卸載的成本結構在兩階段完全不同。以 Llama-3.1-8B 於 126,976 上下文、FP8-KV 計，KV 總量為 7.75 GiB。該平台的 PCIe 為 Gen3  $\times 16$  (15.75 GB/s)，故將全部 KV 移至主機需 492 ms。在 prefill，此成本攤提於 126,976 個 token，等於每 token 0.0024 ms；在 decode，每一步都要重付，而全駐留於 HBM 的同一次讀取僅需 8.3 ms。

| 卸載比例 | decode 每步增加 | 相對全駐留 |
|------|-------------|-------|
| 5%   | +24.6 ms    | 3.0×  |
| 20%  | +98.4 ms    | 11.9× |
| 100% | +492.1 ms   | 59.3× |

在 full attention 下，decode 階段的卸載在任何比例都不可行。即使僅卸載 5%，每步仍付出全駐留 3 倍的代價。唯一可行的路徑是稀疏 attention——僅取回當步 attention 分數高的 block，即 Quest [43]、InfiniGen [25] 與 ArkVale [10] 的做法。

三種工作負載型態的有效動作空間不同。

| 型態         | 決策點                  | 預測的對象                      | 有效動作空間                      |
|------------|----------------------|----------------------------|-----------------------------|
| prefill 為主 | 請求之間保留哪些前綴           | 該前綴是否再被請求（時序）              | 六階全部                        |
| 多輪對話       | 同上，另含思考空窗的機會成本       | 同上                         | 六階全部                        |
| 生成為主       | 請求之內哪些 block 可離開 GPU | 該 block 的 attention 分數（語意） | 僅 GPU 內的精度階；CPU/SSD 慢 3–59× |

**Takeaway.** 第 6 節的實驗涵蓋前兩種型態：其工作負載為「同一前綴送兩次」，prompt 與生成的比例為 1092:1，故 99.8% 的時間落在 prefill。第三種型態的決策點在請求之內，且預測的對象是 attention 分數而非存取機率——那是不同的預測問題，本文不處理。此界線與觀察三的 SSD 結果同構：六階是候選集合，其有效子集由平台與工作負載型態共同決定，而兩者皆可事先算出。

## 2.6 雙平台實驗設計

表 3 的 FLOP/byte 欄位揭示了一個跨越一個數量級的光譜：RTX 3090 為 2,254，MI300X 為 20,752，相差 9.2 倍。此差距的來源是結構性的：3090 的 GDDR6X 頻寬與 PCIe 4.0 鏈路較為接近，而 MI300X 的 HBM3 遠快於其 PCIe 5.0 鏈路。

這使得本文的核心主張成為可證偽的：若最佳 KV 放置策略確實隨硬體比值結構性改變，則同一組策略在相距 9.2 倍的此二平台上應展現不同的最佳動作組合。須說明的是，此二平台位於該光譜的中段而非兩端（表 3 中 GH200 為 9:1、MI355X 為 127:1），我們的可證偽性主張因而受限於實際可取得的硬體跨度。具體而言，我們預

**Table 5:** 雙平台的實驗職責分工。

|         | RTX 3090               | MI300X                       |
|---------|------------------------|------------------------------|
| 記憶體壓力來源 | 24 GB GDDR6X<br>單請求上下文 | 192 GB HBM3<br>並行度／跨 session |
| 主要壓力軸   | context 8K→64K         | 並行度 1→32                     |
| 問題性質    | 容量                     | 機會成本                         |
| 生態      | CUDA (baseline 齊全)     | ROCm(部分受限)                   |
| 能耗量測    | 整卡                     | <b>HBM 分軌</b>                |
| 角色      | 機制驗證 + 完整對照            | 規模 + 主要科學主張                  |

測 Drop→重算動作在 MI300X 上的最佳觸發門檻，應顯著低於在 RTX 3090 上的門檻。

兩個平台因此承擔不同的實驗職責（表 5）。值得強調的是，這並非資源限制下的妥協：單一平台無法檢驗硬體條件化的主張。

**RTX 3090 上的容量分析。** 以 8-bit 權重的 Llama-3.1-8B 計算，24 GB 中約 13.6 GB 可供 KV 使用，對應約 111K token。單請求在 64K 上下文需 7.8 GB（總計 15.8 GB，可置入），而 128K 需 15.6 GB（總計 23.6 GB，超出）。64K 與 128K 之間存在一道明確的容量懸崖，既有文獻所預設的場景在此平台上完全成立。

**CUDA 生態的對照組優勢。** 第 6 節指出 ArkVale、Quest、ShadowKV、ClusterKV 與 RetroInfer 因 rapidsai/raft 缺乏 ROCm 移植而無法在 MI300X 上執行。這些系統在 RTX 3090 上均可執行，雙平台設計同時解決了 AMD 平台上對照組不足的問題。

### 3 Related Work

我們依動作與訊號兩個維度組織既有工作，並在表 6 給出特徵矩陣。判準從嚴：「未來預測」欄僅在該工作預測尚未發生的存取時才標記，僅使用當前步 attention 分數者不計。

#### 3.1 分層儲存

FlexGen [39] 首次以線性規劃決定 KV 在 GPU/CPU/disk 三層間的比例配置，但其策略為離線靜態。CachedAttention [17] 針對多輪對話保留 session 級 KV 於 HBM/DRAM/SSD，並利用排程器佇列進行預取。Mooncake [36] 將叢集中間置的 CPU/DRAM/SSD 池化為共享 KVCache 層。KVDrive [28] 以 prefill 階段的 attention 重要性剖析決定 HBM/DRAM/SSD 的初始配置，並將全量 KV 寫入 SSD 作為後備。Tutti [37] 則專注於 SSD I/O 路徑本身，透過 GPU 發起的儲存存取消除 CPU 瓶頸。

這些工作共同的限制是：放置訊號來自當前或過去的觀測，且不將重算視為動作。

#### 3.2 可恢復驅逐

ArkVale [10] 觀察到 token 重要性隨解碼步驟變動，早期被驅逐者可能重新變得重要，因而以 page digest (key 向量的包絡體) 估計已驅逐頁的重要性並召回。QEvict [19] 則將可恢復性建立在精度維度：中等信心的視窗被量化保存，重新變重要時反量化升回全精度。

這兩類工作分居兩個正交維度：ArkVale 及其後繼者 [42, 43] 做位置分層 (GPU↔CPU，且兩層皆為全精度)，QEvict 做精度分層 (全精度 ↔ 量化，但僅一個位置)。據我們所知，尚無工作以單一重要性驅動策略統一管理兩者。

#### 3.3 學習式未來效用預測

2026 年出現多篇以學習方法預測 KV 未來效用的工作：KVP [34] 以 per-head 的強化學習代理排序 token，其 reward 直接導自未來效用；ForesightKV [11] 建構以真實未來 attention 分數計算的 golden eviction oracle 並以模仿學習逼近；LookaheadKV [4] 則以可學習的 lookahead token 預測模型真實回應所導出的重要性。Marconi [35] 的效用函數同時包含預測重用機率、計算節省與記憶體佔用。

這些工作的動作空間均為單層的保留/丟棄，且無一支援恢復。最強的未來預測器沒有安全網，而最完善的安全網沒有未來預測器。

#### 3.4 重算與傳輸

Cake [23] 讓 GPU 正向計算與 I/O 反向載入同時進行並在中點會合。KVPR [21] 以 profiler 與 scheduler 求解活化值傳輸與 KV 重算的最佳切分。HCache [18] 儲存中間 hidden state (約為 KV 的一半大小) 並由其重建 KV，實質上在階梯上增加了第三階。

這些工作共同的時序特性是檢索時的反應式決策：它們回答「此刻需要這段 KV，如何最快取得」，而非「未來可能需要，現在應置於何處」。ACL 2026 的綜述 [22] 中，整理「GPU KV 重算 ↔ KV 傳輸」重疊的表格僅列出一個範例 (KVPR)。

**以 query 為訊號者無法取代放置決策。** Quest [43]、InfiniGen [25]、ArkVale [10] 與 ShadowKV [42] 皆以當下的 query 向量估計各 block 的重要性並取 top- $k$ 。這是取得單步 attention 稀疏性最直接的訊號，其準確度上限也高於任何以存取歷史為輸入的預測器。然而該類方法在本文的問題上有兩個結構性障礙。

(i) query 只覆蓋當下這一步。 $q \cdot k$  回答的是「此刻這個 query 需要哪些 KV」。跨請求的重用——一段前綴在數分鐘後被另一個尚未抵達的請求命中——所對應的 query 尚不存在，因而無法由此類訊號涵蓋。這正是上一段所述反應式與前瞻式的分界。

(ii) 計算  $q \cdot k$  預設  $k$  仍可取得。要以 query 判斷是否保留某 block，必須先讀取該 block 的 key；而是否保留該 key 正是待決策的變數。此依賴使 query 訊號無法用於已降級至 SSD 或已丟棄的 block，亦即無法用於階梯下緣——而階梯下緣正是成本差異最大之處。以 query 為訊號的方法因此運作於「內容皆可取得，僅需挑選搬運對象」

的前提之上，其前一步（內容該置於何處、是否保留）即為本文所處理的問題。

本文不將二者視為競爭關係：query 側訊號可作為特徵併入前瞻式預測器（第 5.2 節），對仍可取得的 block 提供高品質輸入，而預測器負責覆蓋 query 無法觸及的跨請求與已降級情形。

### 3.5 品質作為約束

KVServe [30] 將壓縮 profile 選擇形式化為  $\arg \min_p T_p(c)$  s.t.  $T_p(c) \leq T_{\text{SLO}} \wedge q_p(w) \geq q_{\min}$ ，是我們所知唯一在 Tier-1 場所將模型品質寫為硬性約束的 KV 工作，但其決策變數不含放置。EvicPress [16] 與 AdaptCache [15] 將品質納入效用函數，但作為可調權重而非約束。LeoAM [26] 是我們調查中唯一給出數值上界者（宣稱準確度下降小於 1%）。OrbitFlow [32] 以 ILP 於延遲 SLO 與記憶體容量約束下決定逐層放置，但因其為無損設計，ILP 中不存在品質變數。

### 3.6 形式化與上界

Fang 等人 [14] 將異質記憶體系統中的 KV 放置問題數學形式化並推導理論上界，明確指出「存在可觀的執行期最佳化空間」，但同樣明確表示不提出具體排程策略。本文正是填補其留下的策略空缺。

### 3.7 特徵矩陣

### 3.8 研究缺口

由表 6 可得三項觀察：

(1) 「重算」與「品質約束」兩欄從未在同一列同時標記。

(2) 四項成本從未同時進入單一線上決策。傳輸成本見於 CacheGen 一系，重算成本見於 Cake/KVPR/HCache，記憶體成本見於 EvicPress，未來效用見於 Marconi/KVP。最高覆蓋為 Marconi 的三項（未來效用、計算節省、記憶體佔用），其因僅在 GPU 上運作而結構性地不可能具備傳輸成本。

(3) 兩條研究血脈互不交會。以 FlexGen 為基礎的一系（IMPRESS、LeoAM、InfiniGen）具備重要性感知與有損處理，但無跨請求重用；以 vLLM 為基礎的一系（CacheBlend [46]、LMCache [2]、Cake、MTDS）具備跨請求重用，但將 KV 視為精確且不可分割。尚無系統同時具備重要性感知的有損分層與跨請求重用。

綜述 [22] 自身的 Observation O7 指出壓縮與放置的協同設計是「a missed opportunity」，Challenge C5 呼籲在共享預算下聯合最佳化 eviction、offload 與 prefetch，而該清單未包含重算。

## 4 Problem Formulation

### 4.1 狀態與動作

設 KV cache 被劃分為 block 集合  $\mathcal{B}$ 。於決策時刻  $t$ ，系統狀態為

$$s_t = \left( \{\ell_i, \pi_i, a_i, n_i\}_{i \in \mathcal{B}}, C^{\text{gpu}}, C^{\text{cpu}}, C^{\text{ssd}}, Q_{\text{pcie}}, \phi_t \right) \quad (3)$$

其中  $\ell_i$  為 block  $i$  的所在層級， $\pi_i$  為其精度， $a_i$  為上次存取時刻， $n_i$  為 token 數， $C^*$  為各層剩餘容量， $Q_{\text{pcie}}$  為互連佇列深度， $\phi_t$  為當前查詢特徵。

動作空間為

$$\mathcal{A} = \{\text{Gpu}_{16}, \text{Gpu}_8, \text{Gpu}_4, \text{Cpu}_8, \text{Ssd}, \text{Drop}\} \quad (4)$$

對應圖 1。Drop 動作意味著該 block 不被儲存於任何位置，若後續被需要則由 token 重算。

階梯是示意，量化器可替換。式 (4) 的六個狀態代表典型部署選項，而非本文主張的特定量化方案。實務上 KV 量化存在一個重要的非對稱性：**key 對量化誤差遠比 value 敏感**，故生產系統普遍採 K/V 分離的精度配置——LMCache 的預設 turboquant\_k8v4 為 key 8 位元、value 4 位元；KIVI [31] 對 key 採 per-channel、對 value 採 per-token 量化，因二者離群值結構不同。

本文的決策框架與此正交：任何量化器只要能給出「容量佔用」與「存取延遲」兩個數字，即可作為階梯的一階。將  $\text{Gpu}_8$  替換為 K8V4、或將  $\text{Gpu}_4$  替換為 KIVI 的 2 位元方案，均不改變式 (8) 的形式，只改變成本常數。

**Drop 的可行性受前序 block 約束。** 其餘五個動作彼此獨立——一個 block 放在哪一層，不影響其他 block 能否被取回。**Drop 是唯一的例外。**

重算 block  $i$  並不需要自 token 0 重跑整個序列：以 chunked prefill 的方式，只需將 block  $i$  的 token 前向通過模型，其 attention 讀取前序 block 既有的 KV。這使重算成本為  $O(|b_i|)$  而非  $O(i)$ 。但這也意味著：

$$a_i = \text{Drop} \text{ 可行} \iff \forall j < i: \text{block } j \text{ 的 KV 可取得} \quad (5)$$

若前序 block 亦被丟棄，必須先重算它們，成本沿依賴鏈累積：

$$\begin{aligned} \text{recomp}(i) &= \sum_{j \in \mathcal{D}(i)} \left( 2N_{\text{active}}|b_j| + \text{attn}(j) \right) \\ \mathcal{D}(i) &= \{i\} \cup \{j < i: a_j = \text{Drop}\} \end{aligned} \quad (6)$$

此約束的意涵是 **Drop 決策彼此耦合，不可獨立最佳化**。一個只看單一 block 效用的策略會系統性地低估連續丟棄的代價。我們以「同一序列內連續 Drop 的 block 數上限」作為可行性約束處理之，並於第 6 節量測該上限對 Pareto 前緣的影響。這也是 HCache [18] 儲存 hidden state 的動機：由 hidden state 可單步重建 KV，從而切斷依賴鏈。

### 4.2 成本模型

對 block  $i$  採取動作  $a$  的即時成本為

$$\begin{aligned} c(i, a) &= \underbrace{\lambda_m \cdot \text{mem}(i, a)}_{\text{容量佔用}} + \underbrace{\lambda_x \cdot \text{xfer}(i, a)}_{\text{搬移成本}} \\ &\quad + \underbrace{p_i \cdot [\text{fetch}(i, a) + \text{recomp}(i, a)]}_{\text{期望存取成本}} \end{aligned} \quad (7)$$

其中  $p_i = \Pr[\text{block } i \text{ 於未來視窗內被存取}]$  為未來效用預測，fetch 為自  $\ell_i$  載回 GPU 的延遲，recomp 僅在  $a = \text{Drop}$  時非零，且依第 2.4 節之模型隨 block 的絕對位置增長。

**Table 6:** 特徵矩陣。● 表示明確支援，○ 表示部分或間接支援。判準從嚴（見正文）。「未來預測」不含僅使用當前步 attention 估計者。

| 工作                | 場所          | GPU/CPU | SSD | 未來預測 | 壓縮 | 恢復 | 重算 | 品質約束 |
|-------------------|-------------|---------|-----|------|----|----|----|------|
| FlexGen [39]      | ICML'23     | ●       | ●   |      | ●  | ●  |    |      |
| InfiniGen [25]    | OSDI'24     | ●       |     | ○    |    | ●  |    |      |
| ArkVale [10]      | NeurIPS'24  | ●       |     |      |    | ●  |    |      |
| Cake [23]         | ICML'25     | ●       | ●   |      |    | ●  | ●  |      |
| HCACHE [18]       | EuroSys'25  | ●       | ●   |      | ○  | ●  | ●  |      |
| Marconi [35]      | MLSys'25    |         |     | ●    |    |    | ○  |      |
| ShadowKV [42]     | ICML'25     | ●       |     |      | ●  | ●  | ○  |      |
| OrbitFlow [32]    | VLDB'26     | ●       |     |      |    | ●  |    |      |
| KVserve [30]      | SIGCOMM'26  | ●       |     |      | ●  |    |    | ●    |
| KVP [34]          | ICML'26     |         |     | ●    |    |    |    |      |
| Het-Mem [14]      | IEEE CAL'25 | ●       |     |      |    | ●  |    |      |
| MTDS [44]         | C&IS'26     | ●       | ●   | ○    |    | ●  | ●  |      |
| KVDrive [28]      | preprint    | ●       | ●   |      |    | ●  |    |      |
| EvicPress [16]    | preprint    | ●       | ●   | ○    | ●  | ●  |    | ○    |
| QEvict [19]       | preprint    |         |     |      | ●  | ●  |    |      |
| <b>Tiara (本文)</b> | —           | ●       | ●   | ●    | ●  | ●  | ●  | ●    |

### 4.3 受約束的最佳化問題

給定品質預算  $\epsilon$ ，

$$\begin{aligned}
& \min_{\{a_i\}} \sum_{i \in B} c(i, a_i) \\
& \text{s.t. } \Delta Q(\{a_i\}) \leq \epsilon \\
& \quad \sum_i \text{mem}^{\text{gpu}}(i, a_i) \leq C^{\text{gpu}} \quad (8) \\
& \quad (\text{對 CPU/SSD 同理}) \\
& \quad \text{TTFT} \leq T_{\text{SLO}}, \text{ TPOT} \leq P_{\text{SLO}} \\
& \quad \text{式 (5) 的可行性約束}
\end{aligned}$$

此形式化與既有工作的差異有三：(i) 動作空間 (4) 同時跨越位置與精度兩個維度，並將 Drop 視為一個具明確標價的層級而非失敗路徑；(ii) 品質為硬性約束而非目標函數中的加權項，此點與 KVserve [30] 一致但作用於放置而非壓縮；(iii) 成本函數 (7) 中四項成本同時出現。

**$\epsilon$  的可操作性。**  $\Delta Q$  於執行期不可直接觀測，這是所有品質約束工作面臨的共同困難。KVserve 採用離線 profiling 加上  $\epsilon$ -greedy bandit 修正。我們規劃在第 6 節評估兩種代理訊號：(a) 離線 profile 建立的 (壓縮率, 任務類型)  $\rightarrow$  品質查找表；(b) 模型輸出分佈的信心值作為線上回饋。此為本設計最主要的風險點，我們於第 7 節說明。

## 5 Design

### 5.1 系統架構

圖 2 給出整體架構。設計上的關鍵決定是：不自行實作推論引擎。vLLM [24] 的 OffloadingConnector 已提供 GPU  $\rightarrow$  CPU  $\rightarrow$  (FS/物件儲存/P2P) 的搬移機制，且官方文件明確載明支援 CUDA、ROCm 與 XPU；其上游 CI 於實體 gfx942 硬體上執行 KV offload 測試。該連接器內建的

**Table 7:** 既有學習式快取／放置預測器的設計比對。最後一欄為本文關注的重點：損失函數是否對不同錯誤方向給予不同代價。

| 系統                | 模型         | 動作數      | 成本不對稱             |
|-------------------|------------|----------|-------------------|
| KVP [34]          | 112×MLP    | 2        | 否                 |
| ForesightKV [11]  | MLP(16)    | 2        | 部分 (單邊)           |
| LookaheadKV [4]   | LoRA $r=8$ | 2        | 否                 |
| Marconi [35]      | 啟發式        | 2        | 否 (但有 value/byte) |
| Mockingjay [38]   | 表格 + TD    | 2        | 否                 |
| LRB [41]          | GBDT       | 2        | 否 (L2)            |
| PARROT [29]       | LSTM+attn  | 2        | 是 (NDCG gain)     |
| Sibyl [40]        | C51 DQN    | $n$      | 是 (實測延遲)          |
| <b>Tiara (本文)</b> | GBDT       | <b>6</b> | <b>是 (動作成本加權)</b> |

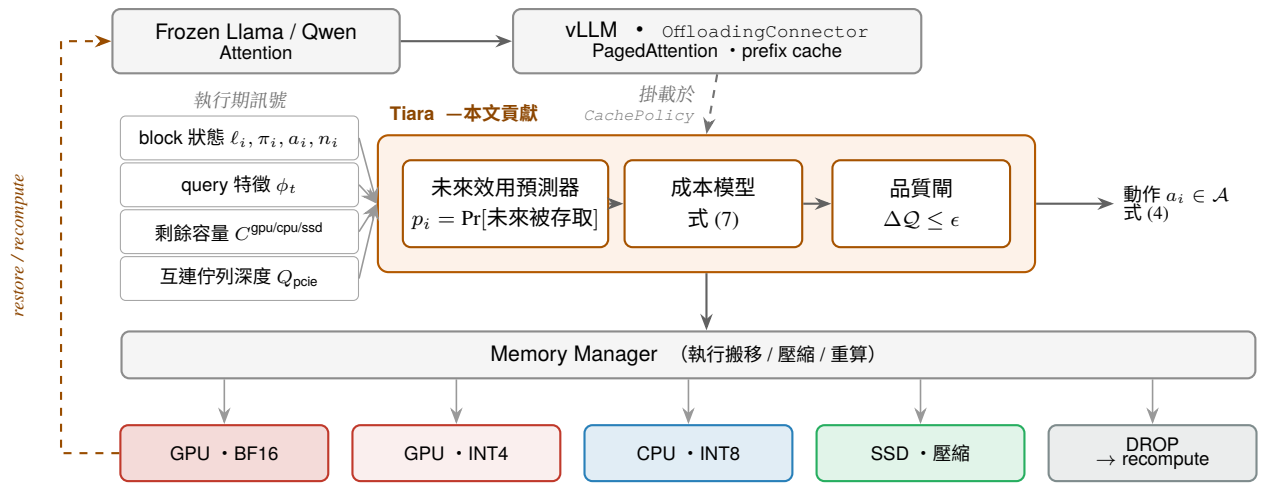
置換策略僅有 lru 與 arc，並透過 CachePolicy 介面開放自訂實作。

此決定有三個後果：(i) Tiara 可在 CUDA 與 ROCm 兩個平台上零移植部署；(ii) 對照組為 production 級實作而非我們自行撰寫的弱化版本；(iii) 貢獻邊界清晰：機制屬於 vLLM，策略屬於本文。

### 5.2 未來效用預測器

預測器輸出  $p_i$ ，即 block  $i$  於未來視窗內被後續請求命中的機率。此定義的範圍須明確： $p_i$  是跨請求的時序量，而非請求之內的 attention 分數。在 full attention 的 decode 階段，每個 block 於每一步皆被讀取， $p_i$  恆為 1，預測無從施力（第 2.5 節）。請求之內的取捨由稀疏 attention 處理，其預測對象為  $q \cdot k$ ，屬不同問題。我們對 8 個既有的學習式快取／放置系統做了設計層級的比對（表 7），並據此做出四項設計決定。

**規格。** 預測器是一個獨立於 LLM 之外的小型模型，不參與 attention 計算，亦不修改模型權重。其介面如下：



**Figure 2: Tiara 架構與資料流。**灰底為既有基礎設施，橘框為本文貢獻。控制器接收四類執行期訊號，經「未來效用預測 → 成本模型 → 品質閾」三階段，為每個 KV block 產生一個動作  $a_i \in \mathcal{A}$ ，交由 Memory Manager 執行。關鍵設計決定是不自行實作推論引擎：Tiara 掛載於 vLLM 既有的 CachePolicy 介面，貢獻邊界因而清晰（機制屬於 vLLM，策略屬於本文），且可在 CUDA 與 ROCm 上零移植部署，並直接與內建的 lru/arc 對照。左側橘色虛線為恢復路徑：被降級或丟棄的 block 在重新變得重要時可升回上層或重算，單次誤判不會造成資訊的永久損失。

|                                                                                                    |                                                           |
|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| <b>輸入</b> （每個 KV block，全部於服務時免費取得）                                                                 |                                                           |
| • <code>deltas[0..31]</code> ：                                                                     | 前 32 次存取的時間間隔（未填滿為 $\infty$ ）                             |
| • <code>edc[0..9]</code> ：                                                                         | 指數衰減計數器，僅於存取時更新                                           |
| • <code>block_size</code> , <code>layer_id</code> , <code>head_id</code> , <code>token_type</code> |                                                           |
| • block 內 key/value 向量的池化統計                                                                        |                                                           |
| • <code>attn_mass[0..7]</code> ：                                                                   | 最近 8 個解碼步中，該 block 取得的 attention 質量佔比（forward 已算出，記錄成本近零） |
| <b>輸出</b>                                                                                          |                                                           |
| $\hat{y}_i = \log \tau_i$                                                                          | （迴歸； $\tau_i$ 為距下次存取的時間）                                  |
| $\downarrow$ isotonic regression                                                                   | （驗證集上校準）                                                  |
| $\hat{p}_i = \Pr[\tau_i \leq W]$                                                                   | （ $W$ 為決策視窗長度）                                            |
| <b>模型</b> ：GBDT，單一全域模型， <code>layer_id/head_id</code> 作為特徵                                         |                                                           |

門檻永遠施加於  $\hat{p}_i$ ，不施加於  $\hat{y}_i$ ——式 (7) 與式 (9) 使用的皆為機率。此區分是必要的：對  $\hat{y}_i$  直接閾值化在數學上不對應式 (9) 的最佳邊界（見第 5.3 節）。

**決定一：預測連續量，而非二元分類。** 所有預測連續量並延後閾值化的系統，都勝過其二元分類的前身，且 Mockingjay [38] 與 LRB [41] 均明確報告此點。LRB 嘗試過對 Belady 邊界的二元分類並依 good-decision-ratio 拒絕之。我們因此回歸  $\log$ (距下次存取的時間)。取對數的理由與 LRB 相同：真正重要的是落在邊界的哪一側，100K 與 2M 的差異遠比 2M 與 3M 重要。

**決定二：特徵取自 LRB 的三個家族，全部在服務時免費。** (i) *deltas*，即距該 block 前  $k$  次存取的時間間隔 ( $k \leq 32$ ，未填滿者為  $\infty$ )，此設計涵蓋 LRU、LRU- $K$  與 S4LRU；(ii) *EDC*，即 10 個指數衰減計數器  $C_i \leftarrow 1 + C_i \cdot 2^{-\Delta_i/2^{9+i}}$ ，僅在存取時更新；(iii) 靜態特徵，即 block 大小、layer id、head id 與 *token* 類型。最後一項的依據是 SAECache [13] 量到不同語意類型 (system prompt / 使用者查詢 / 工具輸出 / 推理鏈) 的重用率差異達 **756 倍**，而該特徵取得成本為零。

**決定二之補充：必須納入模型內部表徵。** 上述特徵全部來自存取歷史，不含模型自身的表徵。這與三個學習式先例形成對比：KVP [34] 明確「僅使用 key 與 value 向量」，LookaheadKV [4] 透過層內 adapter 讀取，Fore-sightKV [11] 蒸餾真實的未來 attention。「在不看 KV 的情況下預測 KV 的重用」作為服務層的低成本選擇是合理的，但作為學習貢獻則站不住腳。我們因此將 block 內 key/value 向量的池化統計併入特徵集，並在第 6 節以消融比較「僅存取歷史」與「存取歷史 + 模型內部表徵」兩種特徵集。

**決定二之補充二：納入 query 側訊號，但作為特徵而非機制。** 第 3 節論證以 query 為訊號者無法取代放置決策，原因是  $q \cdot k$  對已降級的 block 不可得。但對仍位於 GPU 的 block，該訊號的品質高於任何存取歷史特徵，捨棄它並無理由。我們因此記錄 *attn\_mass*，即最近 8 個解碼步中該 block 取得的 attention 質量佔總質量之比。此量在 forward pass 中已被計算，記錄它僅需一次 per-block 的規約，不引入額外的  $q \cdot k$ 。

此設計使特徵集在階梯上呈降級退化的形態：位於 GPU 的 block 同時具備存取歷史與 query 側訊號，位於 CPU/SSD 者僅有存取歷史，已丟棄者僅有靜態特徵。GBDT 原生處理缺失值，此結構無須額外處理。我們在第 6 節以消融量測 *attn\_mass* 的邊際貢獻。

**決定三：GBDT 而非神經網路。** LRB 明確比較過 GBM、線性迴歸、邏輯迴歸、SVM 與 2 層 16 隱藏單元的 NN，GBM 在不同 trace 與快取大小下穩定勝出，且訓練僅需 300 ms、對 64 個候選推論僅需 **30  $\mu$ s**。GBDT 亦能原生處理  $\infty$  值的 delta 特徵。我們採用單一全域模型並以 layer/head id 作為特徵；KVP [34] 的 112 個獨立 agent 帶來可觀的工程成本，而其論文並未提供「per-head 模型優於帶 head-id 特徵的共享模型」的消融證據。

此決定另有兩個本設定特有的理由。(i) 預測器的成本自其收益中扣除。預測器運行於服務熱路徑，其延遲

直接抵銷所避免的重算或傳輸。以表 4 的區間為參照，單次重算為 12–1,972  $\mu\text{s}$ ，而對 64 個候選的 GBDT 推論為 30  $\mu\text{s}$ ；若改以需要一次 forward pass 的模型評分，僅 kernel launch 與排隊即進入相同數量級，收益隨即被抵銷。(ii) 神經預測器與主模型競爭同一加速器。GBDT 於 CPU 上執行，完全不佔用 GPU；任何在 GPU 上評分的預測器都會插入主模型的 decode 批次，其代價不僅是預測器本身的延遲，還包括對批次結構的擾動。

此外，主模型規模上升時此權衡朝有利於 GBDT 的方向移動： $\kappa$  隨模型增大而上升（表 4：8B 至 70B 由 6 增至 21，於平台 A 由 54 增至 190），而 GBDT 的推論成本不隨模型規模改變，因其特徵取自存取歷史而非模型參數。

我們不要求讀者接受此論證，而以實驗回答：表 11 的 (D) 區塊於相同特徵集與相同損失下比較四種預測器，並同時量測品質收益與熱路徑延遲。

前三者（GBDT、小型 MLP、其容量放大一階者）皆接受攤平後的特徵向量，故僅檢驗容量。第四者為直接消費 `deltas` 時間序列的小型 GRU，檢驗的是結構——即時序資訊應由手工特徵編碼（本文與 LRB 的作法），抑或應由模型自行學習。此對照不可省略：LRB 所比較的 2 層 16 隱藏單元 NN 同樣接受攤平輸入，其「GBM 勝出」的結論因而不涵蓋序列模型，本文若僅援引該結論即屬過度推論。

`deltas` 與 EDC 事實上已是時序的手工編碼——前者為存取間隔序列的截斷，後者為 512 至 262,144 十種半衰期上的指數加權和，其功能與循環網路的隱藏狀態同構，差別在於前者的更新規則寫死而後者由訓練得到。(D) 區塊所檢驗的正是此差別是否重要。若 GRU 在計入自身延遲後仍佔優，第 5.2 節的整條特徵工程路線即被推翻，我們將據以修改。

**決定四：標籤取得採用 LRB 的雙機制。** 標籤  $\tau_i$  無法在記錄特徵的當下取得，因其定義涉及未來。我們於取樣時記錄特徵並將該樣本置入待標記佇列，其後由兩個機制之一完成標記：(a) 該 block 下次被存取時，以兩次存取的間隔為標籤；或 (b) 當該樣本離開一個長於 relaxed-Belady 邊界的滑動視窗  $W$  而仍未被存取時，立即以「 $> W$ 」標記。

機制 (b) 是取得負樣本的唯一途徑。僅有機制 (a) 時，訓練集只會包含「後來確實被重用」的 block——從未被重用者永遠等不到第二次存取，因而永遠不進入訓練集。模型將完全未見過應被丟棄的樣本長何模樣，而那正是 Drop 動作的目標群體。取樣以 block 為單位而非以請求為單位，以避免偏向熱門 block。

**決定五：以時間切分而非隨機切分評估。** 訓練與測試集依 trace 的時間順序切分（前 70% 訓練、後 30% 測試），不作隨機切分。隨機切分會使訓練集含有測試期之後的存取事件，而 EDC 與 `delta` 特徵本身即為時間的函數，此洩漏將系統性地高估預測品質。此點在快取預測文獻中屬常識，但其違反不易由結果察覺，我們因此明確記載。

**決定六：以線上重訓因應工作負載飄移。** 不同工作負載的重用結構不同：多輪對話的前綴重複命中，RAG 的文件塊多為一次性讀取，agent 流量則呈現極熱的系統提示與極冷的工具輸出。SAECache [13] 所量到的 756 倍重

用率差異即為此分布差異的直接證據。我們以三個層次因應：(i) `token_type` 作為特徵，使模型得以在單一模型內區分語意類型；(ii) 週期性重訓，利用 GBDT 300 ms 的訓練成本，以最近一段時窗的樣本重新擬合；(iii) 第 6 節報告跨工作負載泛化，即於工作負載 A 訓練並於 B 測試的退化幅度。

第 (ii) 項是 GBDT 選擇的直接後果而非附帶效果。KVP [34] 的 112 個強化學習 agent 需 8 GPU DDP 訓練，其重訓週期以日計，結構上無法追隨以分鐘計的流量變化。對分布飄移的適應成本，是預測器族選擇的一部分，而非可事後補救的工程細節。

### 5.3 成本敏感的損失函數

這是本文與既有工作差異最大的一點，也是我們認為最具貢獻的設計。

**問題：對稱損失在本設定下可證明地失準。** 既有 KV cache 的學習式方法（KVP、LookaheadKV、SP-KV）與經典快取方法（LRB、Mockingjay）的損失函數對兩種錯誤方向給予相同代價。在單一動作（驅逐）的設定下這是合理的。但本文的動作空間有六個成本迥異的選項，兩種錯誤的下游代價相差 6–190 倍，且該比值本身隨平台變動達 32 倍（表 4）：

- **False negative**（預測不會被使用  $\rightarrow$  Drop  $\rightarrow$  後續被需要）：被迫重算，12–108  $\mu\text{s}/\text{token}$
- **False positive**（預測會被使用  $\rightarrow$  保留  $\rightarrow$  實際未使用）：僅浪費容量

令  $\kappa = c_{\text{FN}}/c_{\text{FP}}$  為成本比。保留的期望成本為  $(1 - p)c_{\text{FP}}$ ，丟棄的期望成本為  $p c_{\text{FN}}$ ，兩者相等處即為最佳決策門檻：

$$p^* = \frac{c_{\text{FP}}}{c_{\text{FP}} + c_{\text{FN}}} = \frac{1}{1 + \kappa} \quad (9)$$

這是成本敏感學習的標準結果 [12]，我們不主張其為本文貢獻。其意涵是：在  $\kappa = 10$  時  $p^* = 1/11 \approx 0.09$ 。一個以對稱損失訓練、並在 0.5 處閾值化的預測器，其決策門檻被錯置了約一個數量級。

本文的主張是更具體的一點：既有所有學習式 KV 管理方法的動作空間都是二元且成本同質的，因此  $\kappa = 1$ 、 $p^* = 0.5$ ，對稱損失恰好正確；而本文的動作空間成本異質且  $\kappa$  由硬體決定，對稱損失因而失準。

**做法。** 我們採用兩種形式，並在第 6 節比較：

**(L1) 動作成本加權的 L2。** 在  $\log$  (距下次存取時間) 的迴歸上，令每個樣本的權重等於「策略對該 block 所採取的動作，在判斷錯誤時的代價」：

$$\mathcal{L}_{\text{L1}} = \sum_i w(a_i) \cdot (\hat{y}_i - \log \tau_i)^2, \quad w(a_i) = \frac{\text{cost}_{\text{err}}(a_i)}{\text{cost}_{\text{err}}(\text{Cpu}_g)} \quad (10)$$

此形式是 LRB 的損失加上其不需要的那一項，在 Light-GBM 中僅需 `sample_weight`，無額外機制。

但必須注意一個校準上的限制：對連續目標的迴歸加權，改變的是條件期望  $\mathbb{E}[\log \tau | x]$ ，而非決策門檻。(L1) 本身因而不保證產生式 (9) 的最佳邊界。我們將 (L1) 定位

為啟發式，並在其後串接一個明確的門檻校準步驟：於驗證集上將預測值映射為  $\hat{p}$ （例如以 isotonic regression），再依式 (9) 設定各層級的門檻。

**兩階段做法是正確的基線，本文須證明加權訓練帶來額外增益。** 上述步驟指向一個明顯的替代方案：以對稱損失訓練，僅在決策時將門檻自 0.5 移至  $p^*$ 。此即 Elkan [12] 的結果，且具備一個實際優點—— $\kappa$  隨平台改變時無需重新訓練。若本文的增益全部來自門檻移動，則加權訓練並無貢獻。我們因此在表 11 的 (B) 區塊納入該基線（「對稱 L2 @  $p^*$ 」一列），使兩者可分。

我們預期加權訓練仍有增益，其理由如下。Elkan 的結果成立於機率估計在整個值域上等品質的前提；有限容量的模型並不滿足此前提，它必須決定將擬合能力配置於何處。對稱損失依樣本密度均勻配置，而決策邊界  $p^* = 1/(1 + \kappa)$  隨  $\kappa$  增大而深入機率分佈的尾端：於 MI300X / 8B ( $\kappa = 6$ ) 為 0.14，於平台 A / 70B ( $\kappa = 190$ ) 則為 0.005。在後者，對稱損失幾乎沒有誘因把該區域學準——該處樣本稀少且對平均平方誤差的貢獻極低。加權訓練的作用即是將擬合能力重新配置至該區域。

此論證給出一個可證偽的預測：**(B) 區塊中「對稱 L2 @  $p^*$ 」與 (L1)/(L2) 的差距，應隨  $\kappa$  增大而擴大。**若兩者在 3090 與 MI300X 上差距相同，則加權訓練的正當性不成立，本文將據此縮減主張至門檻校準。

**(L2) 動作成本進入排序損失的 gain 項**（原則性版本）。PARROT [29] 是我們調查中唯一刻意使用非對稱損失的工作，其 NDCG 形式明確地「重罰把機率放在即將被重用的 line 上，輕罰放在遠期重用的 line 上」，並帶來 3.5% 的命中率提升。然而 PARROT 只有單一動作，其 gain 項 ( $d(l_w) - 1$ ) 僅編碼重用距離，未編碼動作成本。我們將動作成本置入該 gain 項：

$$\text{DCG} = \sum_w \frac{\text{cost}_{\text{err}}(a_w) \cdot (d(l_w) - 1)}{\log(\text{pos}(l_w) + 1)} \quad (11)$$

**為何這是可辯護的新意。** Sibyl [40] 是唯一處理異質層級放置且考慮成本的系統，但它迴避了此問題：其 reward 直接取自實測延遲  $1/L_t$ ，成本比是硬體量出來的而非猜的。這是優雅的做法，但 Sibyl 的動作可逆（頁面可在下次存取時重新放置），而本文的 Drop  $\rightarrow$  重算不可逆，因此需要 Sibyl 的驅逐懲罰項  $R_p = 0.001 \times L_e$  推廣為 per-action 懲罰。

**硬體條件化。**  $w(a_i)$  由 FLOP/byte 比值決定，而該比值在 RTX 3090 為 2,254、MI300X 為 20,752（表 3）。同一個損失函數在兩個平台上會產生不同的權重，進而學出不同的策略。第 2.6 節的硬體條件化主張，在此獲得一個具體且可量測的機制，而非僅是敘述。

此處須明確一項取捨。加權訓練使模型綁定於特定的  $\kappa$ ，平台變更即需重訓；上一段的兩階段做法則保持平台無關，但擬合能力的配置不隨決策邊界移動。二者並非優劣關係而是不同的操作點：前者以可移植性換取邊界附近的精度，後者反之。(B) 區塊同時量測二者，使該取捨成為可報告的結果而非未言明的假設。

## 5.4 推論開銷預算

既有工作給出三種可行的攤銷方式：KVP [34] 於 prefill 後評分一次、decode 期間零額外 FLOP（10K 上下文下 0.71 ms，相對於 404 ms 的 prefill）；ForesightKV [11] 每 256 個新 token 重新評分一次，開銷 2.7%；TRIM-KV [8] 於 block 建立時評分一次，之後以指數衰減  $\beta^{t-i}$  免費地重新排序。

我們無法直接採用 TRIM-KV 的「建立時評分一次」，因為第 5.2 節的特徵（deltas 與 EDC 計數器）在每次存取時更新，一次性評分會立即過時。我們改採三層攤銷：(i) 存取時以極低成本更新 EDC；(ii) 於驅逐時機依 LRB 的做法取樣  $k = 64$  個候選評分而非排序整個 cache，該操作在 CPU 上約需 30  $\mu$ s；(iii) 粗粒度的週期性全量重新評分。

線上學習控制器能否符合真實硬體預算，Pythia [7] 提供了存在性證據：其線上 RL 硬體 prefetcher 僅佔 1.03% 的晶片面積。

## 5.5 與現有機制的關係

Tiara 不取代壓縮或稀疏注意力，而是決定何時對哪個 block 施加何種處理。其與 vLLM 的 paged KV 佈局相容，並可與 LMCACHE 的分層後端並存。

## 6 Evaluation Plan

本節為實驗計畫。所有結果表格均為佔位符，實驗尚未執行。

### 6.1 實驗平台

依第 2.6 節的分工，實驗於兩個平台執行。

**平台 A（機制驗證與完整對照）：**NVIDIA RTX 3090（24GB GDDR6X，PCIe 4.0 $\times$ 16），CUDA，vLLM。此平台承擔概念驗證、量測工具鏈建立，以及對 CUDA-only baseline（ArkVale、Quest、ShadowKV、ClusterKV、RetroInfer、ParisKV）與三個學習式 baseline 的完整比較。模型依第 6 節的選模準則為 Qwen2.5-7B-Instruct-1M（主力）與 Llama-3.1-8B-Instruct（對照），均以 AWQ int4 權重載入。

**平台 B（規模與主要科學主張）：**AMD Instinct MI300X（gfx942，192 GB HBM3），ROCm 7.x，vLLM ROCm wheel（Python 3.12），LMCache ROCm wheel（gfx942），真實 NVMe SSD。此平台承擔並行度／機會成本場景與能耗分析。

**可推廣性的界線。** 我們不由平台 A 的結果推論多租戶機會成本場景（24 GB 無法容納多個長 session），亦不由平台 A 提出能耗結論（消費級顯卡缺乏記憶體與計算分軌的能耗計數器）。

### 6.2 壓力軸的設計

壓力軸隨平台而不同，這正是第 2.6 節雙平台設計的直接後果。

**Table 8:** 選模結果。 $M$  為 MoE。 $\kappa$  於各自平台計算。平台 A 的兩個模型  $\kappa$  相差 2 倍，提供同一硬體上的第二個資料點。

| 平台         | 模型                         | $\kappa$ | 角色                 |
|------------|----------------------------|----------|--------------------|
| A (3090)   | Qwen2.5-7B-Inst-1M         | 108      | 主力 (懸崖在 315K)      |
|            | Llama-3.1-8B-Inst          | 54       | 對照 ( $\kappa$ 減半)  |
| B (MI300X) | Llama-4-Scout <sup>M</sup> | 8        | 規模 (原生 10M)        |
|            | Qwen3-30B-A3B <sup>M</sup> | 3        | MoE 對 $\kappa$ 的影響 |

**平台 A (RTX 3090, 24 GB):** 既有文獻的「單請求、上下文遞增」設定在此成立。以 8-bit 權重的 Llama-3.1-8B 計，64K 上下文可置入而 128K 超出，主壓力軸為 context 8K→16K→32K→64K→(128K，預期失敗)。

**平台 B (MI300X, 192 GB):** 同一設定在此不成立：8B–14B 模型於 128K 僅佔 HBM 的 8–10% (第 2 節)，量測不到壓力。此平台改採三軸：

1. **並行度(主軸):** 1、4、8、16、32 個並行請求 @ 128K 上下文。
2. **模型架構:** 見下方選模準則。
3. **跨 session 保留:** 多輪對話與 agentic trace。

**選模準則。** 我們不以「涵蓋愈多模型愈好」為原則，而採一條更嚴格的規則：僅納入會使  $\kappa$  產生可事先計算之變化的模型。據此，選模同時受兩個硬條件約束：

(i) **原生上下文長度必須大於評測長度。** 表 1 顯示 Qwen3 全系列的 `max_position_embeddings` 僅 40,960，於 128K 評測必須施加 RoPE 外推——而外推本身降低品質，將與 KV 管理造成的品質損失混淆，使  $\epsilon$  失去意義。此條件排除了 Qwen3 全系列。

(ii) **必須在目標平台上留有足夠的 KV 預算。** 這排除了在平台 A 上使用 MoE：Qwen3-30B-A3B 即使以 int4 權重載入亦需約 17 GB，僅餘 4.6 GB 供 KV 使用。

據此，兩個平台的模型分工如下：

**Qwen2.5-7B-Instruct-1M 為平台 A 的主力**，因其原生支援 1M 上下文，而其 24 GB 上的容量懸崖落在約 315K token——遠低於模型上限，故量測到的是純粹的記憶體限制而非模型限制。相對地，Llama-3.1-8B 在同平台的容量懸崖 (約 132K) 與其原生上限 (131,072) 幾乎重合，兩種限制無法分離，故僅作對照。

**明確排除的架構。** MLA (如 DeepSeek 系列) 改變式 (1) 本身，其 KV 為壓縮後的潛在向量；hybrid SSM (如 Jamba) 的狀態大小不隨序列成長，已由 Marconi [35] 處理。二者並非「未評估」，而是定義上不同的問題。兩個平台的壓力軸不同並非實驗設計的瑕疵，而是待檢驗的假設本身：若最佳策略確實隨硬體比值改變，則產生壓力的方式本來就應該不同。

**上下文外推的混淆變因。** 選模準則 (i) 已排除所有需要外推的模型，故評測長度均在原生範圍內：Qwen2.5-7B-Instruct-1M 的 315K 容量懸崖遠低於其 1M 原生上限，Llama-3.1-8B-Instruct 的 131K 亦在其原生範圍內。我們因而完全避開了外推造成的品質下降這一混淆變因， $\epsilon$  得

**Table 9:** 成本常數的量測協定。左欄為 block 閒置時持續付出的代價，右欄為被需要時一次性付出的代價。Drop 的價值完全來自左欄的零。

| 動作                | 平時成本 (閒置)   | 被需要時的成本         |
|-------------------|-------------|-----------------|
| Gpu <sub>16</sub> | GPU 位元組     | $\approx 0$     |
| Gpu <sub>8</sub>  | GPU 位元組 /2  | 反量化 kernel      |
| Gpu <sub>4</sub>  | GPU 位元組 /4  | 反量化 kernel      |
| Cpu <sub>8</sub>  | host 位元組 /2 | PCIe + 反量化      |
| Ssd               | 磁碟位元組       | NVMe I/O + PCIe |
| Drop              | 0           | 重算 ( $\ell$ )   |

以純粹反映 KV 管理的影響。若日後需評測超出原生範圍的長度，必須以「full-KV + 相同外推設定」作為品質基準以隔離兩種來源。

### 6.3 成本常數的量測協定

式 (7) 的每一項都需要一組平台與模型相關的常數。這些常數每個 (平台, 模型) 組合只量一次，之後由策略直接查用，不隨每次實驗重量。

關鍵在於**兩類成本必須分開量測**——這正是第 2.4 節論證的核心，若壓成單一維度便會得到「重算最貴故永不重算」的錯誤結論：

**量測方式。** 平時成本以 `torch.cuda.memory_stats()` 與 host 端計數器直接取得，單位為位元組，可由式 (1) 驗算。被需要時的成本則以隔離的微基準取得：對每個動作，反覆搬移或重建固定數量的 block 並取中位數延遲，重複三次確認變異小於 10%。

**重算成本不是常數。**  $\text{recomp}(\ell)$  隨 block 的絕對位置  $\ell$  成長，因 attention 的二次項。我們將其量測為位置的函數而非單一純量，並以線性層的  $2N_{\text{active}}$  項作為  $\ell \rightarrow 0$  的下界檢核。對 MoE 模型， $N_{\text{active}}$  為啟用專家的參數量而非總參數量。

### 6.4 Baselines

表 10 給出 baseline 集合。此處須區分兩種用途不同的對照；混淆二者會使核心主張的檢驗失去判別力。

(i) **檢驗核心主張者為消融，而非外部系統。** 本文主張既有學習式方法所用的對稱損失在異質動作成本下失準。直接比較 Tiara 與 KVP 並不能檢驗此主張：二者在儲存層數 (六階對單階)、預測器族 (GBDT 對強化學習) 與特徵集上皆不同，量到的差異無法歸因至損失函數。具判別力的實驗是固定系統、僅替換損失函數，即表 11 的 (B) 區塊，其中「對稱 L2」一列即為既有方法的損失置於本文動作空間下的行為。該實驗完全在我們控制之下，不依賴任何外部訓練產物。

(ii) **外部學習式系統用於佐證整體競爭力。** KVP、Fore-sightKV 與 LookaheadKV 均已釋出程式碼，但未釋出訓練後的權重，且其預測器為模型專屬 (第 7 節)。我們於平台 A 重訓可得者並回報，未能重訓者引用原論文數值並明確標註未重跑。此列的完整程度不影響 (i) 的結論。

**Table 10:** Baseline 分層與平台可用性。平台 A (CUDA) 上全部可直接執行；平台 B 欄為移植至 ROCm 的估計工作量，經逐一檢視 repo 後得出。<sup>†</sup> 稀疏檢索一系因 rapidsai/raft 無 ROCm 移植而結構性地無法於平台 B 執行——這是平台 A 存在的理由之一。<sup>\*</sup> LMCACHE 在 ROCm 上雖有官方 wheel，但存在已知執行期問題，見第 7 節。ClusterKV、RetroInfer、ParisKV 同屬該系，因與上列三者機制高度重疊而未納入。

| 層級                | Baseline         | A (CUDA) | B (ROCm) |
|-------------------|------------------|----------|----------|
| 錨點                | Full GPU (無卸載)   | ✓        | ✓        |
|                   | Oracle (離線最優)    | ✓        | ✓        |
| Production        | vLLM lru         | ✓        | ✓        |
|                   | vLLM arc         | ✓        | ✓        |
|                   | vLLM + fs 磁碟層    | ✓        | ✓        |
|                   | LMCache [2]      | ✓        | ✓*       |
| 學術 (系統)           | InfiniGen [25]   | ✓        | ~1 週     |
|                   | KIVI [31]        | ✓        | 3-7 天    |
| 學習式               | KVP [34]         | ✓        | 3-7 天    |
|                   | ForesightKV [11] | ✓        | 3-7 天    |
|                   | LookaheadKV [4]  | ✓        | 3-7 天    |
| 稀疏檢索 <sup>†</sup> | ArkVale [10]     | ✓        | ×        |
|                   | Quest [43]       | ✓        | ×        |
|                   | ShadowKV [42]    | ✓        | ×        |

(iii) **production 策略界定實用價值。**vLLM 的 OffloadingConnector 內建策略僅有 lru 與 arc，其餘須經 CachePolicy 介面自行註冊；此二者因而構成目前 production 預設策略的完整集合。兩者皆僅依存取歷史決策：以表 4 的 3090 / Llama-3.1-8B 為例，一個「重算需 225  $\mu$ s」的 block 與一個「自 CPU 取回需 4.2  $\mu$ s」的 block 在 LRU 眼中完全等價。本文的成本模型正是針對此空缺。

**無法於 ROCm 執行者：**ArkVale、Quest、ShadowKV、ClusterKV、RetroInfer、ParisKV。其阻擋原因並非 Flash-Infer (AMD 已維護 ROCm/flashinfer 移植)，而是 rapidsai/raft 無 ROCm 版本，以及 CUTLASS 的 template 層級耦合。我們將引用其論文回報數值作間接對照，並於第 7 節誠實說明。

## 6.5 消融設計

此為本文最重要的實驗。表 11 分為三個區塊，各回應一個不同的質疑。

**(A) 四項成本**回應「把四項成本組合起來是否構成貢獻」。消融以成本項為單位而非以模組為單位：若移除任一項均導致顯著退化，其必要性即為實證結果而非主張。我們亦承認相反的可能：若移除某項不造成退化，正確做法是將其自系統中移除，使貢獻更為聚焦。

**(B) 損失函數**是核心主張的直接檢驗。若成本敏感損失相對於對稱 L2 無可量測優勢，則第 5.3 節的整個論證不成立。此區塊不可省略。

**(C) 機制**為次要，確認品質約束與精度維度確有作用。

**兩平台必須分開報告。**第 5.3 節論證  $w(a_i)$  由  $\kappa$  決定，而  $\kappa$  在兩平台相差 32 倍 (表 4)。(B) 區塊在兩平台上應呈現不同的退化幅度：在  $\kappa$  較大的 3090 上，對稱損失的

**Table 11:** 消融矩陣。每次移除或替換一項，量測 quality-latency-memory Pareto 前緣的退化。(B) 區塊是本文核心主張的直接檢驗。<sup>†</sup>(D) 區塊另行回報熟路徑推論延遲，因較大的預測器須在計入其自身延遲後才可比較。前三列共用攤平特徵，檢驗容量；GRU 一列直接消費 deltas 序列，檢驗結構。任一列在計入自身延遲後仍佔優，第 5.2 節的對應設計選擇即被推翻。

|                                 |                                  | Pareto 退化 |        |
|---------------------------------|----------------------------------|-----------|--------|
| 變體                              | 移除／替換                            | 3090      | MI300X |
| <b>(A) 四項成本</b>                 |                                  |           |        |
| Tiara (完整)                      | —                                | 基準        | 基準     |
| — 未來效用                          | $p_i \rightarrow \text{recency}$ | TBD       | TBD    |
| — 傳輸成本                          | $\lambda_x = 0$                  | TBD       | TBD    |
| — 重算成本                          | 移除 Drop                          | TBD       | TBD    |
| — 記憶體成本                         | $\lambda_m = 0$                  | TBD       | TBD    |
| <b>(B) 損失函數 (核心主張)</b>          |                                  |           |        |
| 對稱 L2 @ 0.5                     | $w \equiv 1$ ，門檻 0.5             | TBD       | TBD    |
| 對稱 L2 @ $p^*$                   | $w \equiv 1$ ，門檻式 (9)            | TBD       | TBD    |
| (L1) 加權 + 校準                    | 式 (9)                            | TBD       | TBD    |
| (L2) 成本進 NDCG                   | —                                | TBD       | TBD    |
| <b>(C) 機制與特徵</b>                |                                  |           |        |
| — 品質約束                          | $\epsilon \rightarrow$ 固定預算      | TBD       | TBD    |
| — 壓縮                            | 移除精度維度                           | TBD       | TBD    |
| — 模型內部特徵                        | 僅存取歷史                            | TBD       | TBD    |
| — query 側訊號                     | 移除 attn_mass                     | TBD       | TBD    |
| <b>(D) 預測器容量與結構<sup>†</sup></b> |                                  |           |        |
| GBDT (本文)                       | —                                | 基準        | 基準     |
| MLP 2 × 64                      | 容量：同特徵、同損失                       | TBD       | TBD    |
| MLP 4 × 256                     | 容量：同特徵、同損失                       | TBD       | TBD    |
| GRU (序列)                        | 結構：直接吃 deltas                    | TBD       | TBD    |

代價應更明顯。若兩平台退化幅度相同，則硬體條件化的主張被否證——這是本文最關鍵的可證偽點。

## 6.6 權重精度的敏感度分析

權重精度不是本文的決策變數，但它改變 KV 可用的預算，因而間接影響最佳策略。我們將其列為敏感度分析而非主要結果：

| 平台         | 權重       | 目的               |
|------------|----------|------------------|
| A (3090)   | AWQ-INT4 | 主要設定 (記憶體受限)     |
| A (3090)   | BF16     | 對照：量化對 KV 預算的影響  |
| B (MI300X) | BF16     | 主要設定             |
| B (MI300X) | FP8      | 對照：CDNA3 之後的常見配置 |

**待檢驗的問題：**在固定的硬體與模型下，最佳 KV 策略是否隨權重精度改變？我們預期會——因為權重精度決定 KV 預算，而預算改變會移動式 (8) 的容量約束。但這是間接影響，與第 2.6 節的  $\kappa$  軸 (直接改變成本比) 性質不同。

**一個須澄清的區分：FP8 運算與 FP8 儲存。**Ampere (sm\_86) 確無原生 FP8 tensor core 運算，但本文動作空間中的 GPU-FP8 是儲存狀態——KV 以 FP8 存放、讀取時反量化——與 tensor core 無關。我們在平台 A 上實

測 `--kv-cache-dtype fp8` 可正常運作：Llama-3.1-8B 的 KV 容量由 41,648 增至 83,312 token、Qwen2.5-7B-1M 由 106,512 增至 213,040 token，二者皆為恰好兩倍且位元組佔用不變。

更進一步，vLLM 亦提供 `int8_per_token_head` 與 `int4_per_token_head`，故動作空間中位於 GPU 上的四個精度階，在平台 A 上皆為同一旗標的不同取值，無須額外實作。以每 GiB 可容 token 數正規化後（消除 KV pool 大小的 run-to-run 抖動，全距降至 0.04%）實測得：

| KV dtype          | KiB/tok | 理想  | 額外           | vs BF16      |
|-------------------|---------|-----|--------------|--------------|
| auto (BF16)       | 128.11  | 128 | +0.11        | 1.00×        |
| fp8               | 64.04   | 64  | +0.04        | <b>2.00×</b> |
| int8_per_tok_head | 66.04   | 64  | <b>+2.04</b> | 1.94×        |
| int4_per_tok_head | 34.02   | 32  | <b>+2.02</b> | <b>3.77×</b> |

fp8 為純格式轉換故恰為 2×；兩個 per-token-head 量化階則各固定多出 2 KiB/token。此開銷可由架構直接算出：Llama-3.1-8B 有 32 層、8 個 KV head，K 與 V 各一份，故每 token 有  $2 \times 32 \times 8 = 512$  個 scale，以 FP32 儲存即  $512 \times 4 = 2,048 \text{ bytes} = 2 \text{ KiB}$ ，與實測相符（int8 為 4.1 bytes/scale、int4 為 4.0）。故精度階梯的收益並非  $2 \times 4 \times$ ，而是  $2.00 \times 1.94 \times 3.77 \times$ ；式 (1) 若僅以每元素位元組數計算，將系統性高估低精度階的收益。量化中繼資料是與序列長度成正比的固定成本，必須進入成本模型。

## 6.7 Oracle 上界與 go/no-go 判斷

我們建構一個離線最優 oracle：給定完整的未來查詢序列，以整數規劃求解式 (8)，得到  $p_i \in \{0, 1\}$  為已知時的最佳放置。此為經典快取文獻中 Belady [6] 分析的直接延伸，並與 Sibyl [40] 報告「達到全知 oracle 的 80%」的評估傳統一致。

Fang 等人 [14] 已為兩層（HBM 與 off-package DRAM）的無損放置推導上界。我們的 oracle 在三點上延伸之：納入 SSD 層、納入 Drop→重算動作、納入品質約束。

**停止條件。** 若 oracle 相對於最佳簡單策略的改善幅度小於 5%，我們將終止此研究方向並報告此負面結果。此判斷於實驗計畫的第 5–6 週執行，早於任何策略實作。

## 6.8 品質約束的兩類動作

式 (8) 的  $\epsilon$  約束對動作空間中的兩類動作意義不同，量法亦須不同。

**無損動作：** $\epsilon = 0$  是可驗證的斷言，不是假設。CPU 與 SSD 僅搬移位元組，理應不改變輸出。此為正確性斷言而非品質取舍：若輸出有異，即為實作錯誤。我們以逐字元比對驗證之——以 64-shot GSM8K（共用前綴 11,029 token）於平台 A 對 60 題取輸出的 SHA-1，與 GPU-BF16 基準比對：

| 設定               | 正確率    | 逐字元相同        |
|------------------|--------|--------------|
| full_gpu（基準）     | 76.67% | —            |
| cpu_lru          | 76.67% | <b>60/60</b> |
| tier_fs（CPU+ 磁碟） | 76.67% | <b>60/60</b> |

位置分層的三階在  $\epsilon$  約束下等價； $\epsilon$  的實質內容全部落在精度分層上。

**有損動作：**目前的樣本數不足以分辨各精度階。我們以同一工作負載量測精度階梯（ $n = 120$ ，貪婪解碼，固定 seed）：

| KV dtype          | 正確率    | 相對 BF16  | $z$  |
|-------------------|--------|----------|------|
| auto (BF16)       | 79.17% | —        | —    |
| fp8               | 73.33% | −5.83 pt | 1.11 |
| int8_per_tok_head | 80.00% | +0.83 pt | 0.16 |
| int4_per_tok_head | 75.83% | −3.33 pt | 0.64 |

三個差值皆不顯著。 $n = 120$ 、 $p \approx 0.79$  時單組標準誤為 3.71 pt，兩組差值的標準誤為 5.24 pt，95% 信賴區間為  $\pm 10.3 \text{ pt}$ ；三個  $z$  值均遠低於 1.96。差值亦非單調（int8 高於 BF16，而 fp8 低於 int4），此非單調性本身即為樣本量不足的徵候。

我們因此不宜稱任何精度階之間存在品質差異。欲以 80% 檢定力分辨 5 pt 的差距（ $\alpha = 0.05$ ）需每組約 1,055 題；GSM8K 測試集有 1,319 題，故此量測可行，將於定稿版本補足。在此之前，表 13 中  $\epsilon$  相關的欄位一律標記為 NOT\_MEASURED。

## 6.9 評估指標與工作負載

系統指標：TTFT、TPOT、端到端延遲、GPU/CPU/SSD 佔用、PCIe traffic、throughput、服務層級分佈（見下）、重算成本。

**ROCm 量測方法。** KV 搬移經由 SDMA copy engine 而非 AQL compute queue，對 `rocprof-compute` 的 `TCC/L2 counter` 不可見。我們將使用 `rocprofv3 --memory-copy-trace` 取得逐筆傳輸的位元組數、方向與時間戳，並以 `rocprof-compute` 的 `gfx942 "Remote Read Traffic"` 指標取得 kernel 側視角。

**能耗。** MI300X 的 `amdsmi` 提供 per-rail 能耗累加器，其中 `hbm_energy_acc` 將 HBM 能耗與計算能耗分離。對一個探討「保留於記憶體或重新計算」的研究問題而言，這是直接對應的量測能力，且在 NVIDIA 平台上不可得。

**品質指標。** Yandex 的研究 [45] 顯示 ShadowKV 等方法在 needle-in-a-haystack 上表現良好卻在多事實抽取任務上失效。因此我們將 needle-in-a-haystack 降級為 sanity check，改以多事實抽取作為主要品質判斷，另納入 LongBench、RULER 與長文本摘要。

**品質如何化約為單一數字。** 表 13 的 Quality 欄須為單一純量，其定義如下。設  $\mathcal{T}$  為上述任務集合， $Q_t(\pi)$  為策略  $\pi$  於任務  $t$  的原始分數，並以 Full GPU 為參照定義相對保留率  $r_t(\pi) = Q_t(\pi) / Q_t(\text{FullGPU})$ 。我們回報

$$Q(\pi) = \min_{t \in \mathcal{T}} r_t(\pi), \quad (12)$$

即最差任務的保留率，而非跨任務平均。理由與上一段相同：平均會使單一任務的崩潰被其餘任務的分數掩蓋，

而 [45] 所報告的正是此類崩潰。品質約束因而具體化為  $Q(\pi) \geq 1 - \epsilon$ 。逐任務的  $r_t$  完整列於附錄，避免式 (12) 的化約隱藏任務間的差異。

**以服務層級分佈取代單一命中率。** 「KV hit rate」對單層快取有明確意義，但本文的動作空間有六階，「命中」並非二元事件：自 CPU 取回與自 SSD 取回的成本相差約一個數量級，與重算則相差 6–190 倍（表 4）。將這些情形壓成單一比率會抹除本文所要量測的成本結構。我們因此回報服務層級分佈，即每次 KV 存取由  $\mathcal{A}$  中哪一階滿足的經驗分佈，並另外給出兩個純量：(i) **GPU 常駐率**，存取時該 block 已位於 GPU 且為 BF16 的比例（對應傳統命中率）；(ii) **重算率**，由 Drop 觸發重算的存取比例。後者為本文特有，直接對應第 2.4 節的成本模型，亦是表 11 中「移除 Drop」一系列的解讀依據。

**預測器層級的診斷：為何不回報分類準確度。** 我們刻意不以預測器的分類準確度作為評估指標，並在此說明理由，以免被誤讀為疏漏。成本敏感訓練的作用即是將決策邊界自 0.5 移至  $p^* = 1/(1 + \kappa)$ ，其必然後果是模型在廉價方向上多犯錯以換取在昂貴方向上少犯錯。以未加權的準確度衡量，成本敏感模型應當劣於對稱模型；若其未劣化，反而意味著加權未生效。準確度因而與本文的目標函數方向相反，不構成有效證據。我們改以兩個量診斷預測器：

(i) **校準品質**。式 (9) 的最佳門檻僅在  $\hat{p}_i$  為校準過的機率時成立：若模型輸出  $\hat{p} = 0.1$  的樣本中實際重用比例並非約 10%，該門檻即失去意義，第 5.3 節的整個論證隨之失效。我們回報 isotonic regression 校準前後的可信度圖與 expected calibration error。此量同時是第 5.3 節所揭露之 L1 缺陷（加權迴歸改變條件均值而非決策門檻）是否被校準步驟補救的唯一直接證據；缺少此圖，該缺陷等均未被處理。

(ii) **成本加權錯誤率**。 $\sum_i c(a_i^{\text{pred}}, a_i^*)$ ，其中  $a_i^*$  為 oracle 動作、 $c(\cdot, \cdot)$  為式 (7) 的成本函數。此量計的是「錯掉多少微秒」而非「錯了幾次」，是唯一能在預測器層級公平比較兩種損失函數的指標，亦與系統層級的端到端延遲直接可對照。

**跨工作負載泛化。** 承第 5.2 節的分布飄移討論，我們額外回報一組交叉實驗：於單一工作負載訓練並於其餘工作負載測試，回報上述兩個診斷量與  $Q(\pi)$  的退化幅度。此結果同時界定週期性重訓的必要性——若跨工作負載退化可忽略，重訓機制即為不必要的複雜度，我們將據以移除。

## 6.10 已完成的量測推翻了本文的四項前提

我們在平台 A 上完成成本常數與容量的量測。四項結果與本文原先的敘述相抵觸，其中兩項擴大了可驗證的範圍，兩項要求修正成本模型。表 12 列出每項量測、其推翻的敘述、以及對論文的後果；量測協定與逐設定的完整數據見附錄 A。

**Takeaway.** 四項結果中，兩項擴大了平台 A 的可驗證範圍（FP8 儲存可用、四個精度階同旗標可切），兩項要求

修正成本模型（量化中繼資料、重算的位置依賴形式）。第四項的性質不同：它不是修正，而是本文核心主張的直接實例——SSD 在平台 A 被 Drop 支配，故六階動作空間並非普遍適用。論文若宣稱六階皆為必要，此結果即為反例；改為「動作空間的有效子集隨平台而定」則與證據一致，且更強。

## 6.11 權重精度決定平台 A 能否進入長上下文區間

BF16 權重在 24 GB 卡上佔 15 GB，KV 預算僅餘 5.9 GB，容量上限 41,648 token。此配置無法進入本文關心的 128K 區間。AWQ-INT4 權重佔 4.7 GB，KV 預算增至約 15 GB：

| 設定                       | KV 容量          | 定址上限    | 有效上限                       |
|--------------------------|----------------|---------|----------------------------|
| Llama-8B BF16 權重         | 41,648         | 131,072 | 41,648                     |
| Llama-8B AWQ 權重          | 120,320        | 131,072 | 120,320                    |
| Llama-8B AWQ + FP8-KV    | 240,656        | 131,072 | <b>131,072<sup>†</sup></b> |
| Qwen-7B-1M AWQ 權重        | 273,872        | 262,144 | <b>262,144<sup>†</sup></b> |
| Qwen-7B-1M AWQ + FP8-KV  | <b>547,744</b> | 262,144 | <b>262,144<sup>†</sup></b> |
| DeepSeek-V2-Lite MLA AWQ | <b>399,376</b> | 163,840 | <b>163,840<sup>†</sup></b> |

<sup>†</sup> 記憶體容量超出模型的可定址長度，瓶頸由硬體轉為架構。

**Takeaway.** 在正確的權重配置下，24 GB 消費級 GPU 的 KV 記憶體不是長上下文的瓶頸；模型的可定址長度才是。六個設定中有四個的 KV 容量超出模型可定址長度，其中 Qwen-7B-1M 於 AWQ 權重加 FP8-KV 下達 547,744 token，為其可定址上限的 2.1 倍。此點對本文的雙平台設計有兩個後果。其一，平台 A 足以驗證 128K 區間的容量與成本主張，無須更大的加速器。其二，MLA 架構將 KV 壓至 30.4 KiB/token（GQA 的 1/4.2），使單卡容量達 399,376 token——單請求的容量壓力於此消失，壓力轉為 PCIe 頻寬與多租戶下的機會成本，即第 2.6 節所述平台 B 的情境。此結果將 MLA 納入本文的  $\kappa$  軸，且在單一平台上提供 4.2× 的跨度。

## 6.12 預期結果（佔位）

## 7 Discussion and Limitations

**本稿無實驗結果。** 這是本稿最主要的限制。第 2 節的分析與第 3 節的比對已完成，但第 6 節的計畫尚未執行。在取得 oracle 結果前，我們無法斷言此問題存在足夠的最佳化空間。

$\epsilon$  是估計值而非保證。品質約束以 profiling 建立的代理訊號執行， $\Delta Q \leq \epsilon$  是經驗性的估計而非形式保證。這是所有品質約束工作的共同限制。將其提升為執行期可驗證的界限（例如以 per-head 的注意力分佈失真上界）是自然的後續方向，但其理論難度高。

**重算可能不划算。** 第 2.4 節的分析顯示重算的單次成本比傳輸高 6–190 倍。若  $p_i$  的預測精度不足，Drop 動作可能持續造成淨損失。消融實驗（表 11）將直接檢驗此點；若 Drop 不帶來收益，我們將自系統中移除之。

**Table 12:** 平台 A (RTX 3090, sm\_86) 的已完成量測。每列給出一項與本文原敘述相牴觸的結果。

| 量測                            | 原敘述 → 實測                                                                                    | 對本文的後果                                                                                                                                  |
|-------------------------------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| FP8 KV 於 sm_86                | 「Ampere 無原生 FP8，平台 A 無此設定」→ 可用，容量恰增 $2.00 \times (41,648 \rightarrow 83,312 \text{ token})$ | 混淆了 FP8 運算與 FP8 儲存。動作空間中位於 GPU 的四個精度階皆為同一旗標的取值，單一平台即可驗證五階                                                                               |
| 精度階梯的實際收益                     | $2 \times / 4 \times \rightarrow 2.00 \times / 1.94 \times / 3.77 \times$                   | per-token-head 量化固定多帶 2 KiB/token 的 scale ( $2 \times 32 \times 8$ 個 FP32)，式 (1) 僅計每元素位元組數會高估低精度階                                       |
| $C_{\text{recompute}}$ 對位置的形式 | 「attention 的二次項」→ 對位置線性，式 (2)，擬合偏差 $< 1.6\%$                                                | 整段 prefill 為 $O(L^2)$ ，但固定大小 block 在位置 $P$ 重算為 $O(C \cdot P)$ 。原敘述混用了兩個量                                                                |
| SSD 與 Drop 的優劣次序              | 「兩者同數量級」→ 成立，且次序隨 block 的絕對位置反轉，交叉點在 7,277 token                                            | Drop 的成本隨位置線性成長而 SSD 為常數，故位置 $< 7,277$ 時 Drop 較廉、其後 SSD 較廉。於 128K 上下文，僅 5.6% 的 block 落在 Drop 一側。 <b>動作空間的有效子集隨上下文長度而變</b> ——此即本文主張的一個實例 |

**Table 13:** 主結果表 (佔位符，實驗尚未執行)

| 方法        | Quality | TTFT | GPU mem |
|-----------|---------|------|---------|
| Full GPU  | —       | —    | —       |
| vLLM lru  | —       | —    | —       |
| vLLM arc  | —       | —    | —       |
| LMCache   | —       | —    | —       |
| InfiniGen | —       | —    | —       |
| Tiara     | —       | —    | —       |
| Oracle    | —       | —    | —       |

**K 與 V 未分離處理。** 式 (1) 前導的 2 對應 key 與 value，本文將二者綁定於同一動作：一個 block 的 K 與 V 一同降級、一同量化、一同丟棄。這是刻意的範圍限制，而文獻指出該粒度並非最佳。KIVI [31] 顯示二者的元素分佈不同，key cache 宜沿 channel 維度量化而 value cache 宜沿 token 維度量化；Hariri 等人 [20] 進一步指出在固定記憶體預算下，將位元優先配置給 key (如 4-bit key 搭配 2-bit value) 較均勻配置嚴格更優；KV-Tuner [27] 則於逐層粒度搜尋混合精度，在 Llama-3.1-8B-Instruct 上達成 3.25 位元的近無損配置。

因此本文階梯上的  $\text{Gpu}_8$  與  $\text{Gpu}_4$  應理解為 K/V 共用精度的保守設定，其品質退化為 K/V 分離配置的上界。將動作空間沿 K/V 拆為兩軸是自然的延伸，但會使  $|A|$  自 6 增至 36，並使標籤生成與 oracle 求解的成本相應上升。本文不涵蓋此延伸；它與本文的核心主張正交——無論動作以何種粒度定義，「動作成本異質時損失函數須編碼該成本」的論點皆成立，且動作粒度變細只會使成本的離散程度更高，因而強化而非削弱該論點。

**ROCm 生態的成熟度。** LMCache 無自動化的 on-device ROCm CI；其預設 MP 模式在 ROCm 上存在已知的死鎖問題；vLLM 在 ROCm 上的 KV 傳輸錯誤目前會導致 worker 終止而非降級 (相關修正 PR 尚未合併)。此外，啟用 KV connector 會使 vLLM 自候選清單移除 `ROCM_ATTN` 後端，我們將記錄每次實驗實際使用的 attention 後端。

**與 AMD 生產堆疊的關係。** AMD 的 ROCm/mori 中的 MORI-UMBP 提供「具分層儲存與分散式鍵值存取的統一記憶體與頻寬池」，已整合入 SGLang HiCache 並具備 SSD 層。我們不宣稱 AMD 平台上缺乏分層 KV 基礎設施；本文的貢獻在於其放置策略，而非分層機制本身。

**平台 A (RTX 3090) 的限制。** 24 GB 的容量意味著平台 A 無法執行單請求 128K 上下文 (第 2.6 節)，亦無法承載多租戶場景，本文不由平台 A 推論機會成本相關結論。此外消費級顯卡缺乏記憶體與計算分軌的能耗計數器，能耗分析僅於平台 B 進行。

**學習式對照組未釋出權重。** KVP [34]、ForesightKV [11] 與 LookaheadKV [4] 三者均未釋出訓練後的權重，且其預測器皆為模型專屬——KVP 需為 Qwen2.5-7B-Instruct 的 112 個 layer-head 各訓練一個 agent (其 README 載明使用 8 GPU DDP)，ForesightKV 需至少兩張 GPU 分置訓練模型與參考模型，LookaheadKV 僅提供「minimal training example」。重訓因而是每個主模型各一次的完整訓練，且主模型變更時須全部重做。

我們因此不以外學習式系統作為核心主張的檢驗手段。本文的主張是「對稱損失在異質動作成本下失準」，而檢驗該主張的正確方式是在同一系統內替換損失函數 (表 11 的 (B) 區塊)，此設計消除了系統間的無關差異。外部學習式 baseline 僅用於佐證整體競爭力，其數值取自原論文並明確標註未重跑。

**無法重跑的對照組。** ArkVale、Quest、ShadowKV、ClusterKV、RetroInfer 與 ParisKV 因 rapidsai/raft 無 ROCm 移植而無法於平台 B 執行；我們於平台 A 執行之，但無法提供這些方法在 MI300X 上的數值。

**MI300A 的架構差異。** 若目標平台為 MI300A (APU) 而非 MI300X，本文的階梯需重新設計。MI300A 的 CPU 與 GPU 共用同一塊 128 GB HBM3，單一 OS 管理的實體位址空間，無 DIMM 亦無主機 DDR [3, 5]。「將 KV 卸載至 CPU 記憶體」在該架構上僅為同一記憶體內的頁表變更，不產生容量收益；階梯將塌縮為 HBM → 對等 APU HBM → NVMe。

## 8 Conclusion

本文的核心論點是：當動作成本相差 6–190 倍、且該比值本身隨硬體變動 32 倍時，以對稱損失訓練的效用預測器會系統性地錯置決策邊界。我們將 KV 放置形式化為受品質約束的序列決策問題，其動作空間統一了位置分層與精度分層，並將「丟棄後重算」視為階梯上一個具明確標價的層級。

平台 A 的量測支持此論點，並在四處修正了本文的敘述(第 6.10 節)。其中兩處擴大了可驗證範圍：FP8 為儲存狀態而非運算狀態，故 sm\_86 可量測，且動作空間中位於 GPU 的四個精度階皆為同一旗標的取值。另兩處要求修正成本模型：per-token-head 量化固定多帶 2 KiB/token 的中繼資料，使精度階梯的收益為  $2.00 \times / 1.94 \times / 3.77 \times$  而非  $2 \times / 4 \times$ ；而固定大小 block 的重算成本對其絕對位置為線性，非二次。

第四處修正的性質不同。SSD 的取回成本為常數 (5.54 ms/block)，而 Drop 的成本隨 block 的絕對位置線性成長(式 (2))，二者於位置 7,277 token 處交叉；其前 Drop 較廉，其後 SSD 較廉。於 128K 上下文，僅 5.6% 的 block 落在 Drop 一側；於 512K 則為 1.4%。這不是本文主張的反例，而是其直接實例：最佳動作空間的有效子集隨上下文長度與平台共同決定，而二者皆可事先算出。六階動作空間應理解為候選集合。

量測亦劃清了品質約束的作用範圍。CPU 與 SSD 的輸出與 GPU-BF16 逐字元相同 (60/60)，故位置分層的三階在  $\epsilon$  約束下等價； $\epsilon$  的實質內容全部落在精度分層上。我們目前的樣本量 ( $n = 120$ ) 不足以分辨各精度階之間的品質差異，故不作此宣稱。

尚未完成的部分決定了本文的下一步。策略的實作與 Oracle 的 go/no-go 判定尚待乾淨環境下的量測；本文的  $\kappa$  跨硬體主張需要平台 B 的數據；而多租戶下的機會成本情境在 24 GB 卡上無法驗證——單卡放不下兩個 128K session。第 6 節給出的判準與第 A.3 節給出的求解方法效力範圍共同約束了這些後續工作可以宣稱什麼。

## Statements

**Data Availability.** 本稿為初稿，尚無實驗資料。第 2 節的 KV footprint 計算工具將隨程式碼一併釋出。所引用模型組態均取自公開的 HuggingFace config.json。

**Ethics.** 本研究不涉及人類受試者或個人資料。工作負載將使用公開 benchmark 與已匿名化的公開對話 trace。

**Author Contributions (CRediT).** (投稿時填入) Conceptualization; Methodology; Software; Investigation; Writing — original draft.

**Conflict of Interest.** (投稿時填入)

**Funding.** (投稿時填入)

**AI Usage Disclosure.** 本稿的文獻調查、比對分析與初稿撰寫過程使用了 AI 輔助研究工具。所有引用文獻均經作者查證其存在性與內容；所有量化分析均可由公開規格與模型組態重現。AI 工具未用於產生實驗數據。

## A 平台 A 的量測明細

本附錄給出第 6.10 節所引數字的量測協定與逐設定數據。所有量測於單張 RTX 3090 (sm\_86, driver 550.163.01) 上以 vLLM 0.28.0+cu129 執行，每列數據可由 run\_id 追溯至指令與輸出檔。

### A.1 量測衛生

該機器由二十餘位使用者共用，此事實決定了三項協定。

**每卡獨佔不足以保證乾淨。** GPU 的 SM 為各卡獨佔，但 PCIe、host RAM 頻寬與 /dev/shm 為全機共用。他人於其他卡上的負載不會出現在本卡的 compute-apps 中，卻會拖慢搬移量測。我們逐格比較序列執行與五卡並行執行的 80 個量測點：不經 PCIe 搬移 KV 的 full\_gpu 差異在  $\pm 2\%$  內，而經 PCIe 的 warm TTFT 差異達  $-26\%$  至  $-52\%$ 。故每列數據記錄整機爭用等級，且僅序列模式的數據進入論文。

**KV pool 大小在同一設定下並非常數。** 同一模型、同一卡、同一 gpu\_memory\_utilization，GPU KV cache size 在 5.09 與 5.88 GiB 兩值間擺動(原始 token 數全距 13.5%)。以每 GiB 可容 token 數正規化後全距降至 0.04%，故容量的平時成本一律以 KiB/token 報告，而非原始 token 數。前一行程結束至驅動釋放記憶體之間存在延遲，我們要求連續三次取樣皆達標才啟動下一次量測。

**分層量測須驗證資料確實落到目標層。** 量測 SSD 時若 CPU 階容量大於工作集，資料不會 cascade 至磁碟。我們的首次量測即因此失效：CPU 階 24 GiB、工作集 8 GiB，vLLM 的 chunk\_queries 顯示磁碟階僅被查詢 2 次而 CPU 階 2,048 次，所得「SSD 與 CPU 僅差 1.2%」實為 CPU 階的數字。將 CPU 階縮至 1 GiB 後磁碟階被查詢 2,048 次，實際讀取 3 GiB。每列數據記錄兩層的 chunk\_queries。

### A.2 成本常數

**平時成本。** 以每 GiB 可容 token 數量測，每設定重複三次取中位數(全距  $\leq 0.04\%$ )：BF16 128.11、fp8 64.04、int8\_per\_tok\_head 66.04、int4\_per\_tok\_head 34.02 KiB/token。後兩者各多出約 2.03 KiB/token，與  $2 \times 32 \times 8 = 512$  個 FP32 scale 的 2,048 bytes 相符(實測每 scale 4.0–4.1 bytes)。

**被需要時的成本。** 以 16K 上下文、四個共用前綴的兩輪工作負載量測 warm TTFT，並扣除 GPU 命中的基準 (134.4 ms)：CPU 736.9 ms、SSD 5,803.3 ms、Drop 5,467.5 ms。換算每 block (16 token) 分別為 0.588、5.536、4.008 ms。

磁碟階的成本並非由 I/O 主導。我們於 SATA SSD (Samsung 870 QVO) 與 NVMe (Crucial P3) 各量一次。裝置頻寬分別為 0.49 與 1.65 GB/s (相差 3.4 $\times$ )，而端到端 warm TTFT 為 5,803 與 6,600 ms——頻寬較高者反而較慢。磁碟讀取僅佔搬移成本的約 35%，其餘為分層機制的查表、cascade 排程與 promotion 開銷。故成本模型應拆為  $C_{\text{ssd}} = C_{\text{lookup}} + C_{\text{sched}} + C_{\text{io}}(\text{device})$ ，僅末項與裝置有關。

### A.3 Oracle 的求解方法與其效力範圍

我們以 trace 驅動的模擬求解上界：GPU 階的逐出以 Bélády/MIN 決定，跨階的目的地以成本感知貪婪決定。後者非加權多階快取問題的最佳解，故所得 headroom 為下界——真正的最佳解只會更好。此不對稱使 GO 判定安全，而 NO-GO 判定須額外審視。

模擬器的效力取決於兩個前提。其一，成本常數須為實測；程式在讀不到 §A.2 的量測檔時中止，不使用預設值。其二，模擬器須能複現已量測的行為；我們以第 6 節的工作負載比對 `full_gpu` 相對 `cpu_lru` 的 warm 段成本比，方向一致 (2/2)，但量級高估約 1.5 $\times$  (實測 9.00 對模擬 14.33)。故本文僅引用 headroom 隨工作負載偏斜的趨勢，不引用其絕對值。

已知的偏差來源為前綴語意：vLLM 的 prefix cache 於中間缺塊時須重算其後全部，而模擬器記為重算單一 block，因而低估 `full_gpu` 的成本。修正此項後 headroom 只會上升。

## References

- [1] NVIDIA ampere GA102 GPU architecture. Whitepaper, NVIDIA Corporation, 2021. v2.1; Appendix A. GeForce RTX 3090: 328 3rd-gen Tensor Cores, 1,695 MHz boost, 71 TFLOPS FP16/BF16 with FP32 accumulate (dense).
- [2] LMCACHE: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665*, 2025. [PREPRINT]; <https://github.com/LMCACHE/LMCACHE>.
- [3] Advanced Micro Devices, Inc. AMD CDNA 3 architecture white paper. Technical report, Advanced Micro Devices, Inc., 2023.
- [4] Jinwoo Ahn, Ingyu Seong, Akhil Kedia, Junhan Kim, Hyemi Jang, Kangwook Lee, and Yongkweon Jeon. LookaheadKV: Fast and accurate KV cache eviction by glimpsing into the future without generation. In *International Conference on Learning Representations (ICLR)*, 2026. Samsung; arXiv:2603.10899.
- [5] Anonymous (see arXiv:2508.12743). Dissecting CPU-GPU unified physical memory on AMD MI300A APUs. *arXiv preprint arXiv:2508.12743*, 2025. [PREPRINT] measurement study.
- [6] L. A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.
- [7] Rahul Bera, Konstantinos Kanellopoulos, Anant V. Nori, Taha Shahroodi, Sreenivas Subramoney, and Onur Mutlu. Pythia: A customizable hardware prefetching framework using online reinforcement learning. In *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2021. arXiv:2109.12021.
- [8] N. Bui, S. Sharma, S. Lamba, S. Mishra, et al. Cache what lasts: Token retention for memory-bounded KV cache in LLMs. *arXiv preprint arXiv:2512.03324*, 2025. [PREPRINT] JPMorganChase AI Research + Yale; venue UNVERIFIED.
- [9] Celestial AI. AMD MI300X GPU performance analysis. *arXiv preprint arXiv:2510.27583*, 2025. [PREPRINT] Celestial AI.
- [10] Renze Chen, Zhuofeng Wang, Beiquan Cao, Tong Wu, Size Zheng, Xiuhong Li, Xuechao Wei, Shengen Yan, Meng Li, and Yun Liang. ArkVale: Efficient generative LLM inference with recallable key-value eviction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. Peking University; Infinigence-AI.
- [11] Zican Dong, Peiyu Liu, Junyi Li, Zhipeng Chen, Han Peng, Shuo Wang, and Wayne Xin Zhao. ForesightKV: Optimizing KV cache eviction for reasoning models by learning long-term contribution. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.03203.
- [12] Charles Elkan. The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 973–978, 2001.
- [13] Shaokai Fang, Ziang Li, Wenfei Wu, Jiatong Ji, Qingsong Liu, and Ruizhi Pu. Not all tokens are worth caching: Learning semantic-aware eviction for LLM prefix caches. *arXiv preprint arXiv:2605.18825*, 2026. [PREPRINT].
- [14] Yunhua Fang, Rui Xie, Asad Ul Haq, Linsen Ma, Kaoutar El Maghraoui, Naigang Wang, Meng Wang, Liu Liu, and Tong Zhang. Accelerating LLM inference via dynamic KV cache placement in heterogeneous memory system. *IEEE Computer Architecture Letters*, 2025. arXiv:2508.13231.
- [15] Shaoting Feng et al. AdaptCache: KV cache native storage hierarchy for low-delay and high-quality language model serving. In *Proceedings of the SOSP Workshop on Big Memory (BigMem)*, 2025. Workshop paper; arXiv:2509.00105.
- [16] Shaoting Feng, Yuhao Liu, Hanchen Li, Xiaokun Chen, Samuel Shen, Kuntai Du, Zhuohan Gu, Rui Zhang, Yuyang Huang, Yihua Cheng, Jiayi Yao, Qizheng Zhang, Ganesh Ananthanarayanan, and Junchen Jiang. EvicPress: Joint KV-cache compression and eviction for efficient LLM serving. *arXiv preprint arXiv:2512.14946*, 2025. [PREPRINT] submitted 2025-12-16.

- [17] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *USENIX Annual Technical Conference (ATC)*, 2024. arXiv:2403.19708.
- [18] Shiwei Gao, Youmin Chen, and Jiwu Shu. Fast state restoration in LLM serving with HCache. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, 2025. arXiv:2410.05004.
- [19] Ayushman Garg, Akshita Gupta, Shaswata Bhattacharya, Abhishek Gupta, Sandeep Kumar, and Manoj Kumar. QEvcit: Recoverable quantized KV eviction for attention-drift-robust long-context decoding. *arXiv preprint arXiv:2608.05326*, 2026. [PREPRINT] submitted 2026-08-05.
- [20] Mohsen Hariri, Alan Luo, Weicong Chen, Shaochen Zhong, Tianyi Zhang, Qifan Wang, Xia Hu, Xiaotian Han, and Vipin Chaudhary. Quantize what counts: More for keys, less for values. *arXiv preprint arXiv:2502.15075*, 2025. [PREPRINT] 2025-02-20.
- [21] Chaoyi Jiang, Lei Gao, Hossein Entezari Zarch, and Murali Annamaram. KVPR: Efficient LLM inference with I/O-aware KV cache partial recomputation. In *Findings of the Association for Computational Linguistics (ACL Findings)*, 2025. arXiv:2411.17089.
- [22] Jiantong Jiang, Peiyu Yang, Rui Zhang, and Feng Liu. Towards efficient large language model serving: A survey on system-aware KV cache optimization. In *Findings of the Association for Computational Linguistics (ACL Findings)*, 2026. arXiv:2607.08057.
- [23] Shuwei Jin, Xueshen Liu, Qingzhao Zhang, and Z. Morley Mao. Compute or load KV cache? why not both? In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. PMLR v267; arXiv:2410.03065.
- [24] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [25] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024. Seoul National University; arXiv:2406.19707.
- [26] LeoAM authors. Breaking the boundaries of long-context LLM inference: Adaptive KV management on a single commodity GPU. *arXiv preprint arXiv:2506.20187*, 2025. [PREPRINT] under review.
- [27] Xing Li, Zeyu Xing, Yiming Li, Linping Qu, Hui-Ling Zhen, Wulong Liu, Yiwu Yao, Sinno Jialin Pan, and Mingxuan Yuan. KVtuner: Sensitivity-aware layer-wise mixed-precision KV cache quantization for efficient and nearly lossless LLM inference. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2502.04420.
- [28] Jian Lin, Jiazhi Mi, Zicong Hong, Haodong Wang, Qianli Liu, Haodyue Zhang, Peng Li, and Song Guo. KV-Drive: A holistic multi-tier KV cache management system for long-context LLM inference. *arXiv preprint arXiv:2605.18071*, 2026. [PREPRINT] submitted 2026-05-18; venue attribution unconfirmed.
- [29] Evan Z. Liu, Milad Hashemi, Kevin Swersky, Parthasarathy Ranganathan, and Junwhan Ahn. An imitation learning approach for cache replacement. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020. PMLR v119.
- [30] Zedong Liu, Xinyang Ma, Dejun Luo, Hairui Zhao, Bing Lu, Wenjing Huang, Yida Gu, Xingchen Liu, Zheng Wei, Jinyang Liu, Dingwen Tao, and Guangming Tan. KVserve: Service-aware KV cache compression for communication-efficient disaggregated LLM serving. In *Proceedings of the ACM SIGCOMM Conference*, 2026. arXiv:2605.13734.
- [31] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.02750.
- [32] Xinyue Ma, Heelim Hong, Taegeon Um, Jongseop Lee, Seoyeong Choy, Woo-Yeon Lee, and Myeongjae Jeon. OrbitFlow: SLO-aware long-context LLM serving with fine-grained KV cache reconfiguration. *Proceedings of the VLDB Endowment (PVLDB)*, 19:1046, 2026. arXiv:2601.10729.
- [33] William Meng, Benjamin Lee, and Hong Wang. Understanding bottlenecks for efficiently serving LLM inference with KV offloading. *arXiv preprint arXiv:2601.19910*, 2026. MLSys 2026 per arXiv comments.
- [34] Luca Moschella, Laura Manduchi, and Ozan Sener. Learning to evict from key-value cache. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. Apple; arXiv:2602.10238.
- [35] Rui Pan, Zhuang Wang, Zhen Jia, Can Karakus, Luca Zancato, Tri Dao, Yida Wang, and Ravi Netravali. Marconi: Prefix caching for the era of hybrid LLMs. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025. Outstanding Paper Honorable Mention; arXiv:2411.19379.

- [36] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — a KV-Cache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST)*, 2025. Best Paper Award; arXiv:2407.00079.
- [37] Shi Qiu, Yifan Hu, Xintao Wang, Wenhao Zhu, Jianqin Yan, Hao Chen, Kaiqiang Xu, Kai Chen, and Yiming Zhang. Tutti: Making SSD-backed KV cache practical for long-context LLM serving. *arXiv preprint arXiv:2605.03375*, 2026. [PREPRINT] submitted 2026-05-05.
- [38] Ishan Shah, Akanksha Jain, and Calvin Lin. Effective mimicry of Belady’s MIN policy. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022.
- [39] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. arXiv:2303.06865.
- [40] Gagandeep Singh, Rakesh Nadig, Jisung Park, Rahul Bera, Nastaran Hajinazar, David Novo, Juan Gómez-Luna, Sander Stuijk, Henk Corporaal, and Onur Mutlu. Sibyl: Adaptive and extensible data placement in hybrid storage systems using online reinforcement learning. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)*, 2022. arXiv:2205.07394.
- [41] Zhenyu Song, Daniel S. Berger, Kai Li, and Wyatt Lloyd. Learning relaxed Belady for content distribution network caching. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 529–544, 2020.
- [42] Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong, Yuejie Chi, and Beidi Chen. ShadowKV: KV cache in shadows for high-throughput long-context LLM inference. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2410.21465.
- [43] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2406.10774.
- [44] Junliang Wang, Jiaqi Hu, Qingping Cao, Yuanrui Zhu, and Xiancheng Lin. Multi-tier dynamic storage of KV cache for LLM inference under resource-constrained conditions. *Complex & Intelligent Systems*, 12(3):104, 2026. Peer-reviewed journal, published online 2026-01-27.
- [45] Yandex Research. KV cache offloading for context-intensive tasks. *arXiv preprint arXiv:2604.08426*, 2026. [PREPRINT]; code at [github.com/yandex-research/context-intensive-kv-offloading](https://github.com/yandex-research/context-intensive-kv-offloading).
- [46] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, 2025. Best Paper Award; arXiv:2405.16444.